

Enjoy the silence: Analysis of stochastic Petri nets with silent transitions

Sander J.J. Leemans^{a,b}, Fabrizio Maria Maggi^c, Marco Montali^{c,*}

^a RWTH Aachen, Germany

^b Fraunhofer, Germany

^c Free University of Bozen-Bolzano, Italy

ARTICLE INFO

Recommended by Matthias Weidlich

Keywords:

Stochastic Petri nets
Stochastic process mining
Silent transitions
Qualitative verification
Markov chains

ABSTRACT

Capturing stochastic behaviour in business and work processes is essential to quantitatively understand how nondeterminism is resolved when taking decisions within the process. This is of special interest in process mining, where event data tracking the actual execution of the process are related to process models, and can then provide insights on frequencies and probabilities. Variants of stochastic Petri nets provide a natural formal basis to represent stochastic behaviour and support different data-driven and model-driven analysis tasks in this spectrum. However, when capturing business processes, such nets inherently need a labelling that maps between transitions and activities. In many state of the art process mining techniques, this labelling is not 1-on-1, leading to unlabelled transitions and activities represented by multiple transitions. At the same time, they have to be analysed in a finite-trace semantics, matching the fact that each process execution consists of finitely many steps. These two aspects impede the direct application of existing techniques for stochastic Petri nets, calling for a novel characterisation that incorporates labels and silent transitions in a finite-trace semantics. In this article, we provide such a characterisation starting from generalised stochastic Petri nets and obtaining the framework of labelled stochastic processes (LSPs). On top of this framework, we introduce different key analysis tasks on the traces of LSPs and their probabilities. We show that all such analysis tasks can be solved analytically, in particular reducing them to a single method that combines automata-based techniques to single out the behaviour of interest within an LSP, with techniques based on absorbing Markov chains to reason on their probabilities. Finally, we demonstrate the significance of how our approach in the context of stochastic conformance checking, illustrating practical feasibility through a proof-of-concept implementation and its application to different datasets.

1. Introduction

The study of stochastic dynamic systems is a long-standing line of research, essential when one wants to quantitatively analyse how agents in AI and business/work processes in BPM resolve the nondeterminism intrinsically present when taking decisions. In the context of Petri nets, seminal works extending Petri nets with stochastic decision making date back to the early 80s [1–3], finally evolving into the framework of *generalised stochastic Petri nets* (GSPNs) [4–6].

In BPM in particular, interest in stochastic processes and their analysis has been recently revived in the context of process mining, exploiting the fact that event data provides the basis for relating behaviour to frequencies and probabilities. This is in turn key to extract useful information for such behaviour: a quality control process with 30% failed checks is a considerably different process than a process with 2% failed checks, even if they have the same control flow. In this light, explicitly enriching process models with a stochastic dimension,

which indicates how likely every model trace is, provides the basis for extracting quantifiable insights, as well as for increasing the quality of further process analysis and mining tasks such as simulation, prediction and recommendation. Not surprisingly, several stochastic process mining contributions have been made in recent times, covering a variety of different tasks:

- discovery of stochastic process models enriched with likelihood information on the different behaviour [7–10];
- process model repair [11];
- stochastic conformance checking, either comparing the stochastic behaviour implicitly represented by an event log with that induced by a stochastic process model [12–16], or considering the likelihood of model traces when aligning the observed with the prescribed behaviour [17].

* Corresponding author.

E-mail addresses: s.leemans@bpm.rwth-aachen.de (S.J.J. Leemans), maggi@inf.unibz.it (F.M. Maggi), montali@inf.unibz.it (M. Montali).

<https://doi.org/10.1016/j.is.2024.102383>

Received 15 July 2023; Received in revised form 14 January 2024; Accepted 21 January 2024

Available online 4 April 2024

0306-4379/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

While all these works employ variants of Petri nets as control-flow backbones of their process, a parallel thread of research infuses stochasticity in declarative process mining [18].

Petri net-based approaches to stochastic process modelling typically come with explicit indications on the relative probability and timing of single steps in the process, which only implicitly yield the overall likelihood of its traces. GSPNs do so by supporting immediate and timed transitions. Immediate transitions have priority over the timed ones, and come with (relative) weights that are used to compute the probability of deciding which immediate transition to fire among the enabled ones. Each timed transition comes with the rate of a corresponding exponential distribution capturing the firing delay of the transition once it becomes enabled. Continuous and discrete Markov chains are traditionally used to analytically address key analysis tasks such as calculating the expected time and probability of evolving a marking into another marking [5,6]. Unfortunately, this well-understood, solid foundational basis cannot be readily applied to solve analysis tasks in BPM and process mining, due to three special requirements when modelling business processes that do not directly match with GSPNs and their execution semantics:

1. *repeatable labels* – transitions in the net must be labelled with corresponding activities in the process, possibly using the same label in multiple transitions;
2. *silent transitions* – there may be transitions that do not correspond to any visible activity in the process, but represent instead internal orchestration steps, such as those capturing gateways in the process;
3. *finite-trace semantics* – the interesting dynamics exhibited by the process are those representing the flow of cases/instances therein, starting from an initial state and reaching one among possibly multiple final states after having executed a finite (yet unbounded) amount of steps, for which only the visible activities are of interest.

The interplay of these three requirements is far from trivial, in particular considering that a single trace of activities may correspond to infinitely many different finite-length paths in the process. This makes it impossible directly to compute the probability of a model trace via enumeration of paths, and explains why stochastic process mining techniques have so far dealt with this problem through approximation [15, 17].

We refer to a *visible* transition if it is not silent. While the firing of a visible transition conceptually captures the execution of its task – this is explicitly recorded as an event in a trace in an event log –, the firing of a silent transition instead indicates the execution of an internal step, not perceived by the external environment, and thus not explicitly recorded. This is useful to capture a number of modelling patterns. We mention two:

- **(control-flow patterns)** silent transitions can be employed to model internal control-flow patterns used to properly orchestrate visible transitions. Examples are skips, loopbacks, split and join points for concurrent branches, and more in general *gateways* in contemporary process modelling notations (such as BPMN).
- **(non-loggable tasks)** a silent transition can replace a visible one when its corresponding task cannot be logged. This is typically the case of a manual activity that is not backed up by a corresponding user-interaction activity, whose purpose is to inform the supporting information system that the activity has been performed.

Silent transitions are also relevant in their full generality in the context of process mining, since many process discovery algorithms produce as output a Petri net with silent transitions [19–21].

To get an intuitive understanding of the problem, consider the labelled stochastic Petri net shown in Fig. 1(a), where silent transitions

are shown in black (we will introduce these nets formally in Section 3). This process has two model traces, but their likelihood may sound counterintuitive. For example, the probability that the process will generate the model trace consisting of a followed by b is $\frac{2}{3}$ [22]. The challenge here stems from the loop of silent transitions, which “favours” b over c. Technically, one can only compute that probability analytically by noticing that the probability of generating a, b corresponds to the sum of infinitely many probabilities, each obtained by executing a different number of iterations within the “silent loop” consisting of the two silent transitions. Specifically, the probability is $\frac{1}{2} + \frac{1}{2} \cdot \frac{1}{4} + \frac{1}{2} \left(\frac{1}{4}\right)^2 + \dots = \frac{1}{2} \sum_{i=0}^{\infty} \left(\frac{1}{4}\right)^i = \frac{\frac{1}{2}}{1-\frac{1}{4}} = \frac{2}{3}$.

Another example of potentially counterintuitive trace probability is shown in Fig. 1(b). In this net, the likelihood of a followed by c is $\frac{3}{4}$. The challenge in this example is again the silent transition, which is used here in a semi-concurrent context: the transition c is mutually exclusive with transition d, but as c is part of two paths (that is, executed before or after the silent transition), its probability is higher than one might expect [23, confusion]. In Section 2, we describe how existing techniques address or circumvent these challenges.

In this article, we start from GSPNs and propose the framework of *(bounded) labelled stochastic processes* (bounded LSPs) to capture processes that incorporate labels and silent transitions in a finite-trace semantics. On top of this framework, we define and analytically solve the following core problems:

1. **Outcome probability:** what is the probability that the bounded LSP evolves from the initial marking to one (or a subset) of its final markings?
2. **Trace probability:** what is the probability of a given trace of the bounded LSP?
3. **Specification probability:** what is the probability that the bounded LSP produces a trace that satisfies a given qualitative specification that captures desired behaviour?
4. **Stochastic compliance:** is the bounded LSP of interest compatible, in behavioural and stochastic terms, to a probabilistic declarative specification [18] indicating which temporal constraints are expected to hold, and with which probability?
5. **Stochastic conformance checking:** how can we employ the previous analysis questions, in particular trace probability, to improve the correctness and applicability of existing stochastic conformance checking techniques [22] relating a reference stochastic process model to a recorded log?

Specifically, we show how outcome probability can be analytically solved by building on and suitably revising the connection [4,6] between GSPNs and discrete-time Markov chains, considering in particular the class of absorbing Markov chains [24, Chapter 11]. We then demonstrate how the other analysis tasks can be reduced to outcome probability, relying on automata-based techniques inspired from those in qualitative verification of Markov chains against temporal properties [25, Ch.10], here adapted to handle finite traces. Such techniques are also used to elegantly deal with silent transitions. As a by-product, we hence provide a solution to an existing problem related to the removal of ϵ -transitions in stochastic finite-state automata, so far only solved through ad-hoc algorithms [26].

Finally, we demonstrate the significance of our approach in the context of stochastic conformance checking, infusing an existing measure [22] with our analytical computation of trace probabilities. We illustrate practical feasibility through a proof-of-concept implementation and its application to different datasets.

This paper is a revised and extended version of [27]. In comparison with [27]:

- We here handle the full features of arbitrary GSPNs.

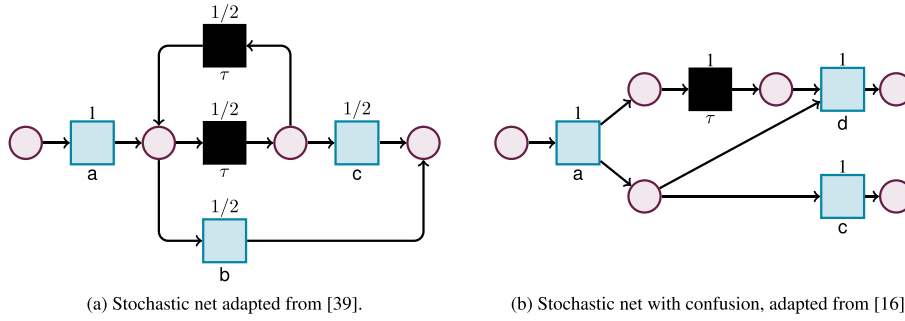


Fig. 1. Two examples of labelled stochastic Petri nets.

- We extend all the foundational results, providing a complete characterisation of the considered analysis tasks, a more explicit connection with Markov chains, and a characterisation of LSPs and their paths through stochastic languages, also examining the impact of livelocks.
- We provide an implementation of the techniques in the ProM framework [28] and an evaluation of stochastic conformance checking techniques, made possible by our technique and its implementation.

The article is organised as follows. Section 2 discusses related work. In Section 3, we introduce the framework of bounded LSPs based on GSPNs and its execution semantics. Section 4 provides a brief roadmap of the different analysis tasks and how they are inter-reduced to each other. In Section 5 we attack the outcome probability problem showing how it can be analytically solved through a connection with absorbing Markov chains. In Section 6 we introduce the qualitative verification of bounded LSPs, show how it can be used to compute the probability of model traces, and reduce it to outcome probability. Section 7 deals with stochastic compliance of bounded LSPs against declarative probabilistic constraints, also in this case providing a reduction to the outcome probability problem. In Section 8 we describe the proof-of-concept implementation of our techniques, demonstrate their application to compute stochastic conformance checking metrics, and report on a corresponding evaluation. Conclusions follow.

2. Related work

Stochastic process-based models have been studied extensively in literature. In the context of this work, we are interested in formal, Petri net-based stochastic models that are at the basis of the recent series of approaches in stochastic process discovery [7,8,10] and conformance checking [14–16,29,30]. Such approaches all refer to the model of (generalised) stochastic Petri nets, or fragments thereof. A first version of this model was proposed in [3], extending Petri nets by assigning exponentially distributed firing *rates* to transitions. This was extended in [4] by distinguishing timed (as in [3]) and immediate transitions. Immediate transitions have priority over timed ones, and have *weights* to define their relative likelihood. As these two types of transitions, abstracting from time, behave homogeneously, we may capture the stochastic behaviour of the net through a discrete-time Markov chain [4].

Several variants of stochastic Petri nets have been investigated starting from the seminal work in [4]. These variants differ from each other depending on the features they support (e.g., arbiters to resolve non-determinism, immediate vs. timed transitions) and the way they express probabilities. Such nets may aid modellers in expressing certain constructs. An orthogonal, important dimension is to ensure that probabilities and concurrency interact properly. This can be achieved through good modelling principles [4,23] or automated techniques [31].

Contrasting these formal models with recent works in stochastic process mining, key differences exist. Traditional stochastic nets do not support transition labels nor silent transitions, and put emphasis on recurring, infinite executions and the so-called steady-state analysis, focused on calculating the probability that an execution is currently placed in a given state. This is done by constructing a discrete-time Markov chain that characterises the stochastic behaviour of the net [3, 4]. Finding the probability of a finite-length trace in such nets is trivial, as every trace corresponds to a single path. However, no transition labels or silent steps are supported, which limits their usefulness for process mining due to the omnipresence of such transitions in process models. On the other hand, when these features are incorporated in stochastic Petri nets, which is precisely what we target in this paper, computing the probability of a trace cannot be approached directly anymore, as infinitely many paths would potentially need to be inspected. At the same time, in business processes we are interested in behaviour at the trace level rather than at the process level – that is, we are not interested in the *state* that a process can be in, but rather on the *path* that a trace follows through the model – thus the large body of work on steady-state-based analyses on Markov models does not apply for our purposes. This explains why reasoning on the stochastic behaviour of such extended nets has been conducted in an approximated way [15, 16], or by imposing restrictions on the model [29].

To bridge this gap, in this paper we take the most basic stochastic Petri nets: we do not consider time or priority, but we add (duplicate) labels and silent transitions. Importantly, *our results seamlessly carry over to bounded, generalised stochastic Petri nets*, thanks to the fact that incorporating priorities in bounded nets is harmless, and that timed and immediate transitions are homogeneous from the stochastic point of view. To the best of our knowledge, outside of recent work using stochastic Petri nets with silent transitions [15,22,29], such nets have not been defined or studied before.

While intuitively stochastic conformance checking techniques need to obtain the probability of a given trace in a stochastic process model (for instance, [22] explicitly obtains this probability to compute a distance measure between a log and a stochastic process model), some stochastic conformance checking techniques avoid computing the probability for a single trace, for instance by playing out the model to obtain a sample of executions [15], or by assuming that the model is deterministic [16]. The results presented in this paper therefore enable the practical application of [22], and may enable further stochastic conformance checking techniques and, consequently, new types of analysis.

Silent steps have been studied in the context of automata. For instance, in [26] an ad-hoc method is described to iteratively remove all silent steps from a stochastic automaton. Due to concurrency and confusion (see for instance Fig. 1(b)), such techniques are not directly applicable to stochastic Petri nets. A result of this paper is that silent steps can be handled directly, without the need for ad-hoc techniques.

3. Labelled stochastic processes based on Petri nets

In this section, we introduce the class of stochastic process models that can be analysed with our techniques. This class essentially builds on (bounded) generalised stochastic Petri nets (GSPNs [6]), where transitions (representing atomic units of work) are [32]:

- *immediate* or *timed*;
- *weighted*, where weights are used to define firing probabilities for immediate transitions, or firing delays for timed ones.

Our model introduces two additional distinguishing features:

1. transitions are *silent* or *visible*, in the latter case denoting *process tasks*, and correspondingly come with a *label* defining the task name;
2. we focus on *finite traces*, progressing process instances through the net from an initial to a final state.

This requires to go through the different features of GSPNs, carefully readapting them to our context.

Before entering into the technical exposition, we give some preliminary notions on multisets. A multiset a over a set U is a function $a : U \rightarrow \mathbb{N}$, where for $u \in U$, $a(u)$ indicates the multiplicity (i.e., the number of occurrences) of u . Set U is called the *support* of the multiset a .

Given two multisets a and b over U , we define:

- the *union* of a and b , denoted by $a + b$, as the multiset that assigns to each $u \in U$ multiplicity $a(u) + b(u)$;
- that a is *included in* b , denoted by $a \leq b$, if for every $u \in U$, we have $a(u) \leq b(u)$;
- assuming $a \leq b$, the *difference* of b and a , denoted by $b - a$, as the multiset that assigns to each $u \in U$ multiplicity $b(u) - a(u)$.

The set of all multisets over U is denoted by $\mathbb{M}(U)$. Multiset a is explicitly represented by, in between squared brackets [...], each element u with non-zero multiplicity, using notation $u^{a(u)}$.

3.1. Labelled Petri nets

We capture the control-flow backbone of a work process using **Petri nets** with **labelled transitions**. Labels are used as follows:

- **(labels as tasks)** labels generally represent (atomic) tasks to be executed within the process;
- **(silent transitions)** a special label is used to indicate that a transition is silent, i.e., does not correspond to any task.
- **(repeated labels)** the same task can label distinct transitions.

To capture transition labels, we assume a given finite set $\mathcal{A}$ of tasks, and a special label $\tau \notin \mathcal{A}$ to indicate a silent step. The whole alphabet of labels is denoted by $\Sigma = \mathcal{A} \cup \{\tau\}$.

We recall the standard definition of labelled Petri nets, where we assume unweighted arcs for simplicity of presentation.

Definition 1 (Labelled Petri net). A (labelled Petri) net N is a tuple $\langle Q, T, F, \ell \rangle$, where:

- Q is a finite set of *places*;
- T is a finite set of *transitions*, disjoint from Q (i.e., $Q \cap T = \emptyset$);
- $F \subseteq (Q \times T) \cup (T \times Q)$ is a *flow relation* connecting places to transitions and transitions to places;
- $\ell : T \rightarrow \Sigma$ is a *labelling function* mapping each transition $t \in T$ to a corresponding label $\ell(t)$ that is either a task name from $\mathcal{A}$ or the silent label τ . $\triangleleft$

We adopt a *dot notation* to extract the component of interest from a tuple. For example, given a net N , its places are denoted by $N.Q$. We will adopt the same notational convention for the other definitions as well. Given a net N and an element $x \in N.Q \cup N.T$, the *preset* and *post-set* of x are respectively denoted by ${}^*x = \{y \mid \langle y, x \rangle \in F\}$ and $x^* = \{y \mid \langle x, y \rangle \in F\}$. If x is a transition, then its pre- and post-set respectively denote its *input* and *output places*.

We now turn to the execution semantics of nets, with the goal of formally describing how they represent the executions of process instances. We start with the notion of state. It is worth noting that our definitions are built on standard notions in Petri nets, such as that of marking, firing, path, reachability graph, but with a key difference: process instances evolve through *finite* sequences of steps (of unbounded length) in the process. Focusing on finite paths and traces differ both conceptually and technically from the typical, infinite-trace execution semantics of Petri nets.

As usual in Petri nets, an execution state of a net is described by a marking. Mathematically, a marking is a multiset of places. Conceptually, it represents a distribution of tokens over places. Each token denotes an execution thread, whose current local state is the place to which the token belongs. All tokens belong to a process instance (also called case), whose current global state is collectively described by the local state of each execution thread.

The distribution of token in a marking determines which transitions are enabled and, in turn, can fire and update the state of the process instance. Specifically, a transition is enabled in a marking if its input places contain at least one token each. Firing an enabled transition produces a new marking where one token per input place is consumed, and each output place gets one token more. This is formally defined using multiset operations as follows.

Definition 2 (Marking). A *marking* m of a net N is a multiset over the places of N , mapping each place $q \in N.Q$ to the number $m(q)$ of tokens in q . $\triangleleft$

Definition 3 (Transition Enablement, Transition Firing). Given a marking m of a net N , a transition $t \in N.T$ is *enabled* in m , written $m[t]_N$, if ${}^*t \leq m$. We denote by $E_N(m)$ the set of enabled transitions in a marking m .

Assuming $m[t]_N$, if t *fires* in m of N , a new marking m' of N is produced, written $m[t]_N m'$, if $m' = (m - {}^*t) + t^*$. $\triangleleft$

Transition firings can be chained in a sequence, obtaining an execution. Before discussing how this is done, we need to introduce a different aspect of transitions: their timing.

3.2. Immediate and timed transitions

Orthogonally to the classification of transitions as visible or silent, we consider a second, temporal dimension, which comes from GNPSs [6]: the distinction between immediate and timed transitions, and the interpretation of weights attached to them.

We start by extending Definition 1 to accommodate these aspects. We stress that our definition presents some minor differences with other definitions of GSPNs in the literature, such as those in [6,32]. First, we do not consider general transition priorities, but we explicitly deal with priorities in the execution semantics. Second, we do not consider inhibitor arcs. This is just for simplicity of presentation; in fact, since we will focus on bounded GSPNs, inhibitor arcs can be seamlessly inserted without any impact on the computational properties of the model. At the same time, differently from the literature, our definition explicitly accounts for transition labels and, in particular, silent transitions, which is a minor change in the definition, but has a major conceptual and technical impact when it comes to the execution semantics and analysis of the stochastic behaviour of these nets.

Definition 4 (*Labelled, Generalised Stochastic Petri Net*). A labelled, generalised stochastic Petri net (LGSPN) N is a tuple $\langle Q, T, F, \ell, w \rangle$, where:

- $\langle Q, T, F, \ell \rangle$ is a labelled Petri net;
- $T = T^{im} \cup T^{ti}$ with $T^{im} \cap T^{ti} = \emptyset$, where T^{im} is the set of *immediate transitions*, and T^{ti} is the set of *timed transitions*;
- $w : T \rightarrow \mathbb{R}^+$ is a *weight function* that assigns a positive number to each transition in N . $\triangleleft$

With some abuse of terminology, we sometimes refer to an LGSPN by implicitly referring to its labelled Petri net.

An immediate transition that becomes enabled fires instantaneously without advancing time, while a timed transition that becomes enabled fires with some delay. The question is then what happens if multiple transitions are enabled.

First and foremost, immediate transitions (being instantaneous) have *priority* over timed ones. This calls for redefining the notion of enablement in Definition 5 as follows.

Definition 5 (*Transition Enablement in a LGSPN*). Given a marking m of a LGSPN N , a transition $t \in N.T$ is *enabled* in m , written $m[t]_N$, if t is enabled in the sense of Definition 5, and either:

- $t \in N.T^{im}$ (i.e., it is immediate), or
- $t \in N.T^{ti}$ (i.e. it is timed), and $N.T^{im} \cap E_m(N) = \emptyset$ (i.e., no immediate transition is enabled in m). $\triangleleft$

If multiple immediate transitions are enabled, one is selected, resolving nondeterminism stochastically using the (relative) weights associated to the enabled transitions. If multiple timed transitions are enabled, one is selected based on a stochastic choice on which transition samples the shortest delay.

More specifically, the weight $w(t) = \lambda$ of a timed transition t denotes the rate of an exponentially distributed random variable describing the probability that, once enabled, the transition samples a certain delay. Technically, consider an exponential random variable X with rate λ , written $X \sim \text{Expo}(\lambda)$, denoting the firing delay of an enabled transition. The probability that such transition fires between a and b time units after becoming enabled is captured by

$$\mathbb{P}(a \leq X \leq b) = \int_a^b f_X(x) dx$$

where

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

is the probability density function of X .

In Section 3.4 we will use weights and rates to define the stochastic semantics of our model, recasting directly the original approach in [6, 32].

Representation of nets. As customary in Petri net literature, we graphically represent GSPNs as bipartite directed graphs with place nodes depicted as circles, immediate transition nodes depicted as segments, and timed transition nodes depicted as squares. Fig. 2 shows the main graphical conventions we adopt in decorating places and transitions with relevant information. The name of a place is indicated below the place circle, while that of a transition is shown inside the transition square, or at the bottom right of the transition segment. The weight (resp., rate) of an immediate (resp., timed) transition is shown on top of the transition icon, while its label is shown at the bottom. Silent transitions labelled by τ are filled in black, while visible transitions labelled by a task from $\mathcal{A}$ are filled in light blue.

Example 1. Fig. 3 shows a LGSPN for a simplified order-to-cash process. Each process instance refers to a distinct purchase order, evolved through visible transitions that capture tasks under the responsibility of

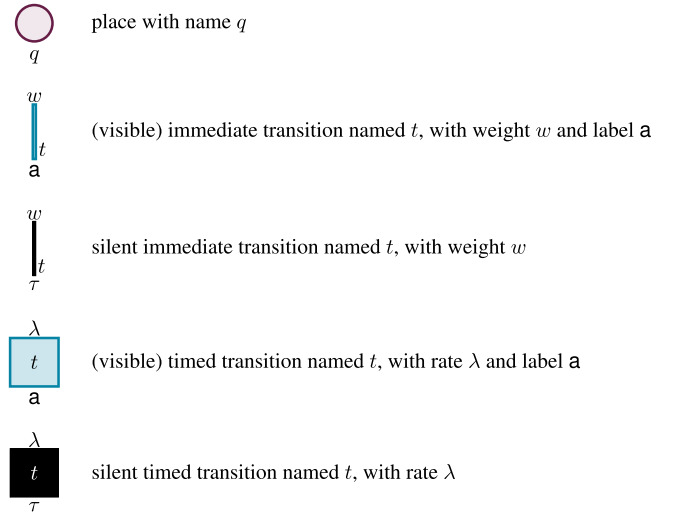


Fig. 2. Notation for GSPNs, considering their three main dimensions: transition labels, silent vs. visible transitions, timed vs. immediate transitions. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

the customer, and silent transitions denoting internal steps of the seller. Consistently with this distribution of responsibility, in a given state the order can be progressed exclusively by the customer or by the seller. We describe next intuitively how the process works.

A new process instance starts when the customer opens an order, and fills it with at least one item. Further items can be then added. Crucially, the seller determines if the customer is actually forced to add further items to progress the order, or whether instead adding further items is just an option. This is a free choice of the seller, which may be concretised in different ways (e.g., requiring certain items to be present together with other items). The first situation is determined when the seller executes the silent transition t_{s_1} (capturing a loopback), the second when the seller executes the silent transition t_{s_2} .

In the latter case, the customer can decide how to continue the process: (i) by adding another item, in turn letting the seller choose whether a further item must or may be added; (ii) by cancelling the order, which leads to complete the process instance in the *cancelled* state (place q_c); (iii) by finalising the order, signalling that the customer wants to proceed to the payment phase. Whether a finalised order actually goes through the payment phase is determined by the seller, who decides whether the order is rejected or accepted. Both decisions have to be acknowledged by the customer. A rejection acknowledgement leads to conclude the process instance placing the order in the *rejected* state (place q_r).

An acceptance acknowledgement gives control back to the customer, who can decide how to proceed: (i) by adding another item, which brings the order back to the previous phase, where it can be finalised, cancelled, or filled with further items; (ii) by cancelling the order, which leads, as before, to complete the process instance in the *cancelled* state (place q_c); (iii) by paying the order (which we assume always succeeds for simplicity). Upon payment, two tasks are executed by the seller concurrently (i.e., in no particular order): a receipt is emitted for the customer, and the order leaves the warehouse for delivery. Once both tasks are executed, the process instance successfully complete in the *happy*, final state (place q_h). Notice the usage of the two silent transitions t_{s_5} and t_{s_6} to respectively split and join the flow of control around the two concurrent tasks emit receipt and ship.

Throughout this example, transitions denoting tasks are all timed, whereas internal moves used for process orchestrations are captures as silent, immediate transitions. Weights and rates are represented symbolically. $\triangleleft$

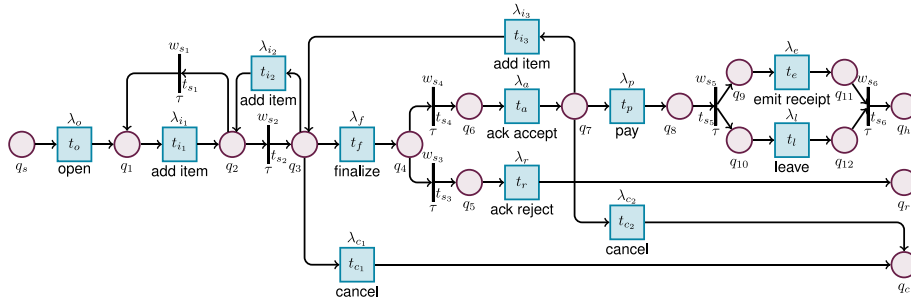


Fig. 3. Stochastic net of an order-to-cash process. Weights and rates are presented symbolically.

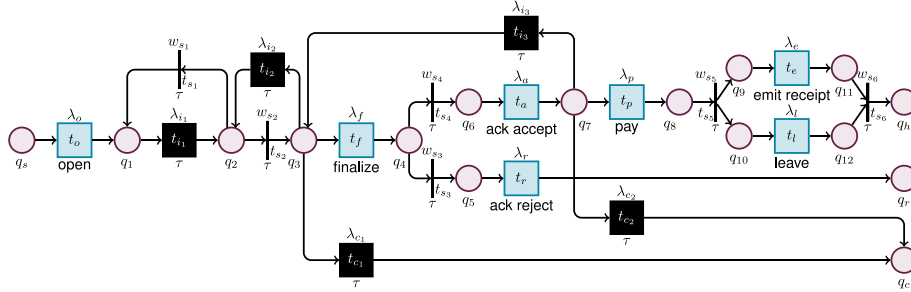


Fig. 4. Stochastic net obtained from that of Fig. 3 in the case where the activities add item and cancel are not logged.

The LGSPN shown in Fig. 3 only contains visible timed transitions and silent immediate ones. As shown next, this is not mandatory.

Example 2. Consider an information system supporting the execution of the net shown in Fig. 3, in a setting where the addition of items and the cancellation of orders cannot be logged (for example because they are manual tasks executed by the customer without interacting with the information system itself). To account for this key aspect, the two tasks add item and cancel order should be replaced by τ , turning the corresponding transitions into silent ones. This leads to the LGSPN shown in Fig. 4. Notice that the newly introduced silent transitions are all timed, as they denote the execution of non-logged tasks, each introducing some delay upon progressing the execution of a process instance. $\triangleleft$

3.3. Labelled stochastic processes and their finite-trace execution semantics (without probabilities)

We now use LGSPNs as a basis for our model of *labelled stochastic processes*, and describe their execution semantics based on finite traces. For the moment, we consider choices as purely nondeterministic, while in Section 3.4 we handle their stochastic behaviour.

Specifically, we are interested in using LGSPNs to describe the execution of process instances. This is done by fixing an initial marking describing the initial state of the process, and by executing the net starting from this initial marking and culminating in one of the final states of the process, which we define to be all deadlock markings.

Definition 6 (Deadlock Marking). A marking m of an LGSPN N is a *deadlock marking* if no transition is enabled in it: $E_N(m) = \emptyset$. $\triangleleft$

Definition 7 (Execution, Supporting Marking Sequence). Let N be an LGSPN, and let m_s and m_f be two markings of N . An *execution* of N from m_s to m_f is a (possibly empty) finite sequence $t_0, \dots, t_n$ of transitions in $N.T$ such that there exists a corresponding sequence of markings $m_0, \dots, m_{n+1}$ of N satisfying the following conditions: (i) $m_0 = m_s$, (ii) $m_{n+1} = m_f$, (iii) for every $i \in \{0, \dots, n\}$, we have $m_i[t_i]_N m_{i+1}$. We call the (unique) sequence $m_0, \dots, m_{n+1}$ the *supporting marking sequence* of $t_0, \dots, t_n$. $\triangleleft$

Definition 8 (Labelled Stochastic Process). A *labelled stochastic process (LSP)* is a triple $\langle N, m_0, M_f \rangle$, where:

- N is an LGSPN representing its *supporting net*;
- m_0 is a marking of N representing the *initial marking*;
- M_f is a finite set of deadlock markings of N representing its *final markings*. $\triangleleft$

Example 3. From now on, we will consider, as a running example, the LSP $\mathcal{N}_{\text{order}}$ defined as follows:

- its supporting net $\mathcal{N}_{\text{order}}.N$ is the LGSPN shown in Fig. 4;
- its initial marking $\mathcal{N}_{\text{order}}.m_0$ is the marking $[q_s]$;
- its final markings $\mathcal{N}_{\text{order}}.M_f$ are the three markings $[q_h]$, $[q_r]$, and $[q_c]$, representing the happy completion of the process where the order is paid and shipped, and the unsuccessful completions where the order is either rejected or cancelled. $\triangleleft$

We will interchangeably assign markings to nets and to LSPs, simply meaning the following: a marking of LSP $\mathcal{N}$ is a marking of $\mathcal{N}.N$.

Paths are defined as LSP executions linking the initial state to one of the final states.

Definition 9 (Path through an LSP). A path through an LSP $\mathcal{N}$ is an execution (as per Definition 7) that starts in the initial marking $\mathcal{N}.m_0$ of $\mathcal{N}$ and ends in one of its final markings $\mathcal{N}.M_f$. $\triangleleft$

Example 4. Consider the LSP $\mathcal{N}_{\text{order}}$ from Example 3. The transition sequence

$$\eta_{\text{fair}} = t_o, t_{i1}, t_{s2}, t_f, t_{s4}, t_a, t_{i3}, t_f, t_{s3}, t_r$$

is a path through $\mathcal{N}_{\text{order}}$, leading from the initial marking $[q_s]$ to the final, rejection marking $[q_r]$. It does so by adding an item to the order, passing once through acceptance, then adding another item, finally being rejected.

The transition sequence:

$$\eta_{\text{ilair}} = t_o, t_{i1}, t_{s2}, t_{i2}, t_{s2}, t_f, t_{s4}, t_a, t_{i3}, t_f, t_{s3}, t_r$$

is another path through $\mathcal{N}_{\text{order}}$, which resembles η_{fair} with the only difference that two items are added before the first acceptance (later turned in a rejection). $\triangleleft$

When logging paths, silent transitions disappear, while visible ones leave their corresponding task as a footprint. We capture this by lifting paths to traces: each trace $\sigma = e_0, \dots, e_n \in \mathcal{A}^*$ is a sequence of events over $\mathcal{A}$ where, for simplicity, each event e_i simply tracks the task that has been executed, together with its qualitative position in the trace with respect to the other events. A trace is a *model trace* for an LSP if it is produced by one of its paths, stripping off the silent transitions and considering the labels of the visible ones, keeping their relative positioning.

Definition 10 (Model Trace, Induced Trace). A trace σ is a *model trace* of LSP $\mathcal{N}$ if there exists a path $\eta = t_0, \dots, t_m$ of $\mathcal{N}.N$ whose corresponding sequence of labels $\mathcal{N}.N.\ell(\eta) = \mathcal{N}.N.\ell(t_0), \dots, \mathcal{N}.N.\ell(t_m)$ coincides with σ once all τ elements are removed. In this case, we say that η induces σ . $\triangleleft$

The following remark highlights one of the main phenomena arising when considering silent transitions. This will become a key challenge when reasoning over LSPs and their stochastic behaviour.

Remark 1. A model trace σ of a (bounded) LSP $\mathcal{N}$ can in general be induced by multiple, possibly infinitely many, paths through $\mathcal{N}$. $\triangleleft$

We denote the (possibly infinite) set of paths through LSP $\mathcal{N}$ that induce trace σ by $\text{paths}_{\mathcal{N}}(\sigma)$. The following example substantiates [Remark 1](#).

Example 5. Consider the two paths η_{fair} and η_{fair} from [Example 4](#). They both induce the following trace of $\mathcal{N}_{\text{order}}$:

$\sigma_{\text{ar}} = \text{open, finalize, ack accept, finalize, ack reject}$

In fact, the addition of items is always handled via silent transitions, hence being completely invisible at the trace level.

More in general, trace σ_{ar} is induced by the infinitely many paths captured through the following regular expression:

$$\text{paths}_{\mathcal{N}_{\text{order}}}(\sigma_{\text{ar}}) = \left\{ \eta \mid \eta \text{ matches } \left(\begin{array}{l} \text{open; (add item)}^+; \text{finalize;} \\ \text{(ack accept; (add item)}^+; \text{finalize)}^*; \\ \text{ack reject} \end{array} \right) \right\}$$

where (i) the atomic regular expression is a task name from $\mathcal{A}$; (ii) expression “ $e_1; e_2$ ” is the concatenation of expression e_1 followed by expression e_2 ; (iii) “ e^* ” is the Kleene star-closure of expression e , capturing the set of regular expressions where e is repeated zero or more times; (iv) “ e^+ ” captures the set of expressions where e is repeated at least once. $\triangleleft$

The execution semantics of an LSP is described by a (possibly infinite-state) labelled transition system typically named a *reachability graph*. In our setting, labelled transition systems are deterministic, in the sense that given a state and a label, there can be at most one transition having that state as source and that label attached. This will be essential when turning to the stochastic behaviour of LSPs.

Definition 11 (Labelled Transition System, path). A (*deterministic*) labelled transition system Λ is a tuple $\langle \mathcal{L}, S, s_0, S_f, \rho \rangle$ where:

- $\mathcal{L}$ is a finite set of labels;
- S is a (possibly infinite) set of states;
- $s_0 \in S$ is the initial state;
- $S_f \subseteq S$ is the set of accepting states;
- $\rho : (S \times \mathcal{L}) \rightarrow S$ is a *transition function*, that is, a partial function that, given a state $s \in S$ and a label $l \in \mathcal{L}$, is either undefined or returns a single successor state $s' = \rho(s, l)$;

- ρ is undefined for states in S_f (that is, final states to not have successors).

A *path* through Λ is a finite (possibly empty) sequence $l_0 \dots l_n$ such that there exists a corresponding sequence $r_0, \dots, r_{n+1}$ of states in S satisfying the following properties: (i) $r_0 = s_0$; (ii) $r_{n+1} \in S_f$; (iii) for every $i \in \{0, \dots, n\}$, we have that $\rho(r_i, l_i)$ is defined, and $\rho(r_i, l_i) = r_{i+1}$. $\triangleleft$

When defining the execution semantics of an LSP, the resulting reachability graph is a labelled transition system with the following characteristics: (i) labels are built from the LSP transitions; (ii) states correspond to markings of the LSP that are reachable from the initial state of the LSP; (iii) initial and accepting states correspond to initial and final markings of the LSP; (iv) transitions match transition firings of the LSP, keeping track of the transition name for provenance. The distinction between labels of the LSP (task names or τ), and those of the reachability graph (transition names paired with their label) is key to guaranteeing determinism of the reachability graph itself. In fact, labelling the transition system with the labels of the LSP would introduce nondeterminism.

Definition 12 (Reachability Graph). The *reachability graph* $RG(\mathcal{N})$ of an LSP $\mathcal{N}$ is a labelled transition system $\langle \mathcal{L}, S, s_0, S_f, \rho \rangle$ with $\mathcal{L} = \mathcal{N}.N.T$ and whose other components are defined by mutual induction as the minimal sets satisfying the following conditions:

1. $s_0 = \mathcal{N}.m_0, s_0 \in S$;
2. for every state $m \in S$, every transition $t \in \mathcal{N}.N.T$ with label $l = \mathcal{N}.N.\ell(t)$, and every marking $m' \in \mathbb{M}(\mathcal{N}.N.Q)$, if $m[t]_{\mathcal{N}.N} m'$ we have that (a) $m' \in S$; (b) if $m' \in \mathcal{N}.M_f$, then $m' \in S_f$; (c) $\rho(m, t) = m'$. $\triangleleft$

It will be useful later to refer to outgoing transitions from a given state s . We do so by defining: $\text{succ}_{RG(\mathcal{N})}(s) = \{s' \mid RG(\mathcal{N}).\rho(s, (t, l)) = s' \text{ for some } t, l\}$.

Given an LSP $\mathcal{N}$, the paths through $RG(\mathcal{N})$ (in the sense of [Definition 11](#)) match with all and only the paths through $\mathcal{N}$ (in the sense of [Definition 9](#)). In addition, due to our requirement that all final markings are deadlock markings, accepting states of $RG(\mathcal{N})$ have no outgoing transitions either.

In general, $RG(\mathcal{N})$ may contain infinitely many states, if it is possible to unconditionally fire transitions to create bigger and bigger markings. When capturing work processes, this is undesired, as the control flow characterising the possible evolutions of each single process instance is expected to be finite-state. In Petri net terms, this means that work processes should be represented by **bounded** nets, as defined next.

Definition 13 (Bounded LSP). An LSP $\mathcal{N}$ is *bounded* if there exists a number k such that, for every reachable marking $m \in RG(\mathcal{N}).S$ and every place $q \in \mathcal{N}.N.Q$, we have $m(q) \leq k$. $\triangleleft$

As desired, a bounded LSP induces a reachability graph that has finitely many states. Boundedness is a standard property assumed when capturing work processes. Verifying boundedness is decidable for Petri nets [33] and well-known techniques exist. It is instead undecidable for GSPNs, due to the implicit priority of immediate transitions over timed ones, but becomes decidable if these two types of transitions are separated, in the following sense: considering transition enablement only based on the presence of enough tokens in the transition input places, in every reachable marking it is never the case that both some timed and some immediate transition are enabled. The LGSPNs of [Figs. 3 and 4](#) indeed satisfy this separation.

Since a bounded LSP reaches finitely many distinct markings, it also reaches finitely many distinct deadlock ones. We then single out the class of complete bounded LSPs, which exhaustively consider all their deadlock markings as final ones.

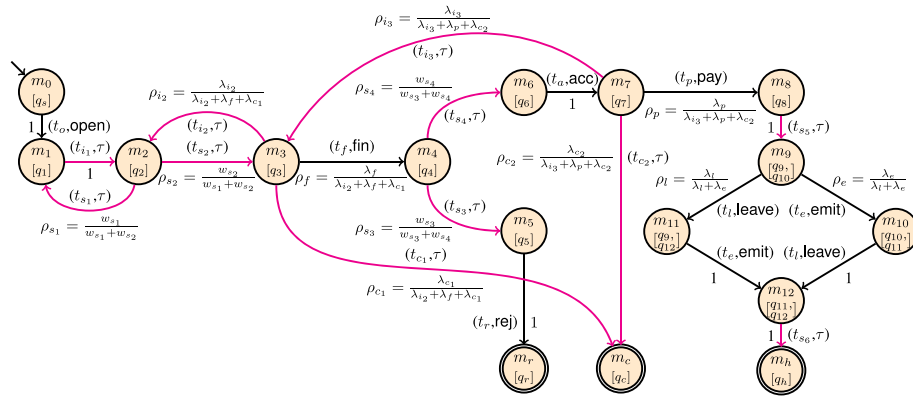


Fig. 5. Stochastic reachability graph of the order-to-cash LSP. States are named. The initial state is shown with a small incoming edge. Final states have a double contour. To ease readability, in addition to transition names we also include the corresponding task names (or τ). Some task names are shortened, and we depict transitions that correspond to the firing of silent net transitions in magenta. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

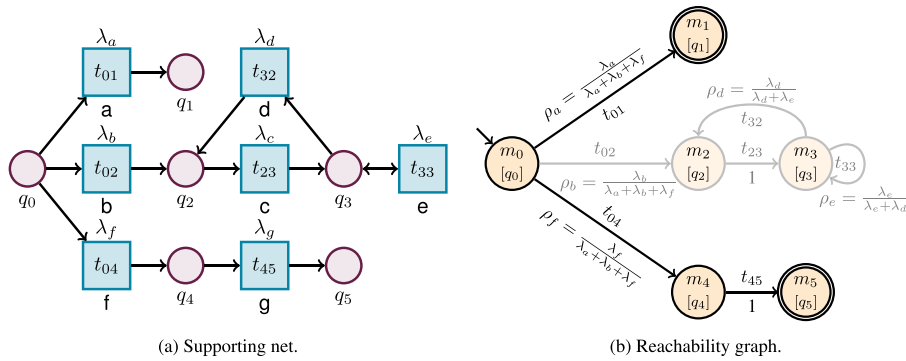


Fig. 6. Reachability graph (b) of an LSP with supporting net shown in (a), initial marking $[q_0]$ and final markings $[q_1]$ and $[q_5]$. Markings s_2 and s_3 are livelock markings. The portion of the transition system that cannot reach a final marking is greyed out.

Definition 14. A bounded LSP $\mathcal{N}$ is *complete* if the set $\mathcal{N}.S_f$ of its final markings coincide with the set of deadlock markings in $RG(\mathcal{N})$. $\triangleleft$

In the remainder of this paper, we focus on bounded LSPs, always implicitly assuming that they are complete. This is just for convenience, and we remark that all the technical results presented hereafter seamlessly carry over the whole class of bounded LSPs.

Example 6. Without considering, for the moment, ρ -attachments on edges, Fig. 5 shows the finite reachability graph of the LSP $\mathcal{N}_{\text{order}}$, which is 1-bounded. It also witnesses that $\mathcal{N}_{\text{order}}$ is a complete bounded LSP. $\triangleleft$

We close this section by identifying undesired reachable markings that cannot be progressed to reach one of the final markings of the LSP of interest. They will be of special interest when reasoning over bounded LSPs and their stochastic behaviour. Such markings resemble the well-known notion of *livelock*.

Definition 15 (Livelock Marking). A marking $s \in RG(\mathcal{N}).S$ of a bounded LSP $\mathcal{N}$ is a *livelock marking* if it is not a deadlock marking, and there is no execution that starts from s and leads to some deadlock marking. $\triangleleft$

Since bounded LSPs have finitely many reachable markings, the only possibility of creating livelocks is to recur over the same set of markings, without ever having the possibility of exiting from the loop to reach a deadlock marking.

The following example illustrates these notions.

Example 7. Consider the LSP $\mathcal{N}_{\text{live}}$ of Fig. 6. The LSP contains an initial choice. If one of the two transitions t_{01} and t_{04} is selected, the

process instance completes in the only final marking. If t_{02} is selected, the process instance enters into a part of the process where it is forced to progress forever - a livelock situation. By looking at the reachability graph, it is in fact clear that the LSP contains two livelock markings - when a token is assigned either to q_2 or q_3 . $\triangleleft$

3.4. Labelled stochastic processes and their stochastic behaviour

We finally extend LSPs with stochastic behaviour, by incorporating **stochastic decision making to determine which enabled transition to fire**. Technically, given a marking, we want to obtain a probability distribution over the transitions that are enabled therein, and use it to choose which one is fired next. This is where transition weights come into play [32].

The resulting execution semantics is then defined through a stochastic transition system.

Definition 16 (Stochastic Transition System). A *stochastic transition system* is a tuple $\langle \mathcal{L}, S, s_0, S_f, \rho, p \rangle$ where $\langle \mathcal{L}, S, s_0, S_f, \rho \rangle$ is a labelled transition system (in the sense of Definition 11), while $p : \rho \rightarrow [0, 1]$ is a *transition probability function* mapping each transition in ρ to a corresponding probability value in $[0, 1]$, such that for every non-deadlock marking $s \in S$, $\sum_{\xi=(s_1, t, s_2) \in \rho \text{ s.t. } s_1=s} p(\xi) = 1$. $\triangleleft$

To define the stochastic execution semantics of LSP, we then need to indicate how to enrich its reachability graph with a transition probability function. For immediate transitions, weights are directly used to define the relative likelihood that one among the enabled transitions will fire: each firing probability is in fact simply obtained as the ratio between the weight of the corresponding transition and the sum of weights of all enabled transitions.

An essential aspect of GSPNs is that exactly the same probability distribution is obtained also in the case of timed transitions. We briefly recall why this is the case. Consider two exponential random variables $X \sim \text{Expo}(\lambda)$ and $Y \sim \text{Expo}(\mu)$. We are interested in computing the probability that the delay sampled by X is shorter than that sampled by Y , which in turn means that the transition associated by X will *fire first*. By recalling Section 3.2, noticing that the two exponential distributions are independent from each other, and that they are 0 for negative values, we recall the well-known result (see, e.g., [34, Chapter 2]) that $\mathbb{P}(X \leq Y) =$

$$\begin{aligned} & \int_0^\infty \int_x^\infty f_X(x) f_Y(y) dy dx = \int_0^\infty \int_x^\infty \lambda e^{-\lambda x} \mu e^{-\mu y} dy dx \\ &= \int_0^\infty \lambda e^{-\lambda x} \int_x^\infty \mu e^{-\mu y} dy dx = \int_0^\infty \lambda e^{-\lambda x} \left[-e^{-\mu y} \right]_x^\infty dx \\ &= \int_0^\infty \lambda e^{-\lambda x} e^{-\mu x} dx = \int_0^\infty \lambda e^{-(\lambda+\mu)x} dx = \left[-\frac{\lambda}{\lambda+\mu} e^{-(\lambda+\mu)x} \right]_0^\infty = \frac{\lambda}{\lambda+\mu} \end{aligned}$$

Consequently, we get the following approach to compute the firing probability, which homogeneously applies to immediate and timed transitions.

Definition 17 (Firing Probability). Let $\mathcal{N}$ be an LSP and m be a marking of $\mathcal{N}$. The *firing probability* of a transition $t \in \mathcal{N}.N.T$ in m , written $\mathbb{P}_{\mathcal{N}}(t \mid m)$, is defined as:

$$\mathbb{P}_{\mathcal{N}}(t \mid m) = \begin{cases} \frac{w(t)}{\sum_{t' \in E_{\mathcal{N}}(m)} w(t')} & t \in E_{\mathcal{N}}(m) \\ 0 & \text{otherwise} \end{cases}$$

The execution semantics of an LSP is then defined by extending its reachability graph with a transition probability function based on firing probabilities.

Definition 18 (Stochastic Reachability Graph). The *stochastic reachability graph* $SRG(\mathcal{N})$ of an LSP $\mathcal{N}$ is a stochastic labelled transition system $\langle \mathcal{L}, S, s_0, S_f, \rho, p \rangle$ where $\langle \mathcal{L}, S, s_0, S_f, \rho \rangle = RG(\mathcal{N})$, and p is defined as follows: for every transition $\xi = \langle m, t, m' \rangle \in \rho$, we have that $p(\xi) = \mathbb{P}_{\mathcal{N}}(t \mid m)$. $\triangleleft$

Example 8. Consider the LSP $\mathcal{N}_{\text{order}}$ from Example 3, whose supporting net is shown in Fig. 4. Fig. 5 captures the stochastic reachability graph $SRG(\mathcal{N}_{\text{order}})$ (using the symbolic weights/rates shown in Fig. 4). If we fix the weight $w(t_{s_4})$ of transition t_{s_4} to 80, and the weight $w(t_{s_3})$ of transition t_{s_3} to 20, we are representing that, whenever a decision has to be taken on whether to accept or reject an order, then there is 0.2 chance that the decision is of rejection, and 0.8 chance that the decision is of acceptance. $\triangleleft$

A final step in the definition of the execution semantics of LSPs, is to lift firing probabilities to the level of paths and traces. As customary in GSPNs, since every firing contained in a path corresponds to an independent choice, we simply need to compute the product of the firing probabilities.

Definition 19 (Path Probability). The probability that a LSP $\mathcal{N}$ produces a path $\eta = t_0, \dots, t_n$ of $\mathcal{N}$, called the *path probability* of η according to $\mathcal{N}$, is:

$$\mathbb{P}_{\mathcal{N}}(\eta) = \prod_{i=1}^n \mathbb{P}_{\mathcal{N}}(t_i \mid m_i)$$

where $m_0, \dots, m_{n+1}$ is the supporting marking sequence for η (cf. Definition 7). The same notion is equivalently defined given as input a stochastic transition system. $\triangleleft$

The probability of a trace is then simply obtained by summing up the probabilities of all the paths inducing that trace.

Definition 20 (Trace Probability). The probability that a LSP $\mathcal{N}$ produces a model trace σ of $\mathcal{N}$, called the *trace probability* of η according to $\mathcal{N}$, is:

$$\mathbb{P}_{\mathcal{N}}(\sigma) = \sum_{\eta \in \text{paths}_{\mathcal{N}}(\sigma)} \mathbb{P}_{\mathcal{N}}(\eta)$$

The central problem we are now facing, which will be tackled in the second part of the article, is that due to Remark 1, this is in general an infinite sum, even in the case of bounded LSPs.

Example 9. Consider the model trace σ_{ar} of $\mathcal{N}_{\text{order}}$ from Example 5. The trace probability $\mathbb{P}_{\mathcal{N}_{\text{order}}}(\sigma_{ar})$ cannot be directly computed, as it requires to sum the probabilities of the infinitely many paths $\text{paths}_{\mathcal{N}_{\text{order}}}(\sigma_{ar})$. $\triangleleft$

4. Analysis of bounded stochastic labelled processes

Now that we have settled the formal model of bounded LSPs, we are ready to tackle their analysis. We recall from the introduction the five fundamental analysis tasks we tackle: outcome probability, trace probability, specification probability, stochastic compliance and stochastic conformance checking. We address these challenges as follows:

- In Section 5, we formalise the problem of **outcome probability** and show how it can be solved using *state-of-the-art analytic techniques for the analysis of absorbing Markov chains* [24].
- In Section 6, we formalise the problem of **specification probability** and show how it can be reduced to that of **outcome probability**; we do so by employing *state-of-the-art qualitative model checking techniques for Markov chains* [25], infused with a *technique to handle silent transitions* and suitably defined over finite traces. We then argue that traces can be encoded as specifications, hence reducing **trace probability** to **specification probability**.
- In Section 7, we introduce the problem of **stochastic compliance**, and show how it can be solved via **specification probability**.
- In Section 8, we describe a *proof-of-concept implementation* to solve the specification probability problem following the technique explained in Section 6, and put the implementation into practice by showing how **stochastic conformance checking** measures can be *effectively computed on a number of real-life logs*.

5. Computing outcome probabilities

We tackle the problem of computing outcome probabilities, that is, determining the probability that a process instance of the bounded LSP of interest evolves from the initial to one (or a subset) of the final markings, representing the *outcome(s) of interest*. For example, we may be interested in knowing the probability that the bounded LSP $\mathcal{N}_{\text{order}}$ of our running example (Fig. 3) evolves an order from *opening* (marking $[qs]$) to *happy completion* (marking $[qh]$).

Formally, we define the problem as follows:

Problem 1 ($\text{OUTCOME-PROB}(\mathcal{N}, F)$).

Input: bounded LSP $\mathcal{N}$, non-empty set $F \subseteq \mathcal{N}.M_f$;

Output: probability value $\mathbb{P}_{\mathcal{N}}(F) = \sum_{\eta \text{ path through } \mathcal{N} \text{ ending in some } m \in F} \mathbb{P}_{\mathcal{N}}(\eta)$. $\triangleleft$

The same problem can also get, as input, a stochastic transition system instead of a bounded LSP.

OUTCOME-PROB cannot be solved through an enumeration of paths, as there may be infinitely many paths reaching the desired marking(s). So far, approximated enumerations have been used, by fixing a maximum threshold either on the length of paths [15], or on their minimum probability [17].

5.1. Absorbing Markov chains

To obtain an analytically exact answer, we build on the direct connection, known from literature [3,4], between GSPNs and discrete-time Markov chains [25,34]. This is possible since if one does not consider labels, stochastic transition systems (in the sense of Definition 16) are in fact discrete-time Markov chains. In the literature, the Markov chain corresponding to the reachability graph of a bounded GSPN is called the *embedded/jump* chain, which can be seen as the core of the continuous-time Markov chain capturing the timed execution semantics of the net, once delays are stripped away and one is only interested in the firing probabilities [3,4]. Forgetting labels has another effect: it blurs the distinction between silent and visible transitions. Hence, the same approach used in the literature for bounded GSPNs can be employed, in spirit, to deal with bounded LSPs. There is a main difference, though: we have to suitably mirror the finite-trace semantics of a bounded LSP into its corresponding Markov chains. We do so by establishing a connection between bounded LSPs and the class of *absorbing Markov chains* [24, Chapter 11].

Instead of using the classical definition of Markov chains via probability transition matrixes, we define them via their graph representation, which provides a direct correspondence with stochastic transition systems as per Definition 16.

Definition 21 (Markov Chain, Walk, Walk Probability). An *absorbing Markov chain* is a tuple $\mathcal{M} = \langle V, \rho, p \rangle$ where:

- V is a finite set of *states*;
- $\rho \subseteq V \times V$ is a *transition relation*, such that $\langle V, \rho \rangle$ is a *connected graph*;
- $p : \rho \rightarrow [0, 1]$ is a *transition probability function* mapping each transition in ρ to a corresponding probability value in $[0, 1]$, such that for every state $v \in V$, we have $\sum_{\xi=(v_1, v_2) \in \rho \text{ s.t. } v_1=v} p(\xi) = 1$.

A *walk* over $\mathcal{M}$ from state $v_0 \in V$ to state $v_n \in V$ is a sequence $w = v_0 \dots v_n$ of states, such that for every $i \in \{0, n-1\}$, we have $\langle v_i, v_{i+1} \rangle \in \rho$. The probability of w is $\mathbb{P}_{\mathcal{M}}(w) = \prod_{i=0}^{n-1} p(\langle v_i, v_{i+1} \rangle)$. $\triangleleft$

Definition 22 (Absorbing Markov Chain, Proper Walk). An *absorbing Markov chain* is a Markov chain $\mathcal{M} = \langle V, \rho, p \rangle$, where the set V of states is partitioned into two subsets V^a and V^t :

- V^a is the set of *absorbing states*, so that $v \in V^a$ if and only if the only transition having v as source is the self-loop $\langle v, v \rangle$ (thus implying $p(\langle v, v \rangle) = 1$);
- $V^t = V \setminus V^a$ is the set of *transient states*, so that $v \in V^t$ if and only if there is a walk from v to some absorbing state in V^a .

A *proper walk* over $\mathcal{M}$ is a walk from a transient state $v \in V^t$ to an absorbing state $v_f \in V^a$, which does not recur over v_f (i.e., the previous-last state of the walk is different from v_f). $\triangleleft$

The notion of proper walk helps establishing a direct correspondence with paths through an LSP. Notice that having longer walks recurring over the same absorbing state would not change at all their overall probability.

Within the wide spectrum of analysis tasks for Markov chains, the main problem we are interested in is that of calculating the *absorption probability*. Given a transient state v and a set A of one or more absorption states, the absorption probability of v is the probability that a walk from v ends up in one of the states A . This corresponds to the overall probability of proper walks from v to one of the states in A .

Problem 2 (ABSORPTION-PROB($\mathcal{M}, v, A$)).

Input: Absorbing Markov chain $\mathcal{M}$, transient state $v \in \mathcal{M}.V^t$, non-empty set $A \subseteq \mathcal{M}.V^a$ of absorbing states;

Output: probability value $\mathbb{P}_{\mathcal{M}}(A|v) = \sum_w \text{proper walk from } v \text{ to some } v_f \in A \mathbb{P}_{\mathcal{M}}(w)$. $\triangleleft$

This problem can be analytically tackled by solving a system of linear equations, which encode the step-wise semantics of the input absorbing Markov chain, and how probabilities are propagated therein. In particular, every state $v_i \in \mathcal{M}.V$ gets a corresponding state variable x_{v_i} , capturing the probability $\mathbb{P}_{\mathcal{M}}(A|v_i)$ of reaching one of the absorbing states in A from state v_i ; this means that variable x_v represents the solution of the problem. Then, each equation defines the value of a state variable x_{v_i} as follows:

Base case (targeted absorbing state) if $v_i \in A$, that is, v_i is one of the targeted absorbing states, then $x_{v_i} = 1$;

Base case (non-targeted absorbing state) if $v_i \in V^a \setminus A$, that is, v_i is a non-targeted absorbing state, then $x_{v_i} = 0$;

Inductive case if v_i is a transient state, its variable is equal to sum of the state variables of its successor states, each weighted by the corresponding transition probability to move to that successor.

Formally, ABSORPTION-PROB($\mathcal{M}, v, A$) leads to the following system $\mathcal{E}_{\mathcal{M}}(v, A)$ of linear equations, defined as follows:

$$x_{v_i} = 1 \quad \text{for each } v_i \in A \quad (1)$$

$$x_{v_j} = 0 \quad \text{for each } v_j \in \mathcal{M}.V^a \setminus A \quad (2)$$

$$x_{v_k} = \sum_{\langle v_k, v'_k \rangle \in \mathcal{M}.\rho} \mathcal{M}.p(\langle v_k, v'_k \rangle) \cdot x_{v'_k} \quad \text{for each } v_k \in \mathcal{M}.V^t \quad (3)$$

We will introduce examples of systems of this kind in the next section. To conclude this introductory discussion on absorbing Markov chains, we recall the following central result.

Theorem 1 (From [24]). Given an absorbing Markov chain $\mathcal{M}$, a transient state $v \in \mathcal{M}.V^t$, and a non-empty subset A of absorbing states from $\mathcal{M}.V^a$, we have:

1. $\mathcal{E}_{\mathcal{M}}(v, A)$ has exactly one solution;
2. ABSORPTION-PROB($\mathcal{M}, v, A$) coincides with the value of state variable x_v extracted from the unique solution of $\mathcal{E}_{\mathcal{M}}(v, A)$. $\triangleleft$

5.2. Connecting ltps to (absorbing) Markov chains

We now encode stochastic transition systems and LSPs into corresponding Markov chains. The encoding is straightforward: transition labels are stripped off, and final states get a self-loop with probability 1, thus becoming absorbing states in the chain.

Definition 23. The embedded Markov chain of a stochastic transition system $A = \langle \mathcal{L}, S, s_0, S_f, \rho, p \rangle$ is the Markov chain $Chain(A) = \langle V, \rho', p' \rangle$, where:

- $V = S$;
- $\rho' = \rho \cup \{ \langle s, s \rangle \mid s \in S_f \}$;
- $p' = \{ \langle s_1, s_2 \rangle \mapsto p(\langle s_1, \ell, s_2 \rangle) \mid \langle s_1, \ell, s_2 \rangle \in \rho \} \cup \{ \langle s, s \rangle \mapsto 1 \mid s \in S_f \}$.

The embedded Markov chain of a bounded LSP $\mathcal{N}$ is the Markov chain $Chain(\mathcal{N}) = Chain(RG(\mathcal{N}))$. $\triangleleft$

By inspecting the notion of reachability graph of a bounded LSP, as well as the notion of livelock marking, we immediately get the following important result.

Lemma 1. For a livelock-free bounded LSP $\mathcal{N}$, we have that $Chain(\mathcal{N})$ is an absorbing Markov chain. $\triangleleft$

Proof. From Definition 23, we have that the final markings of $RG(\mathcal{N})$, which are deadlock markings and hence do not have any successor, get encoded by $Chain(A)$ into absorbing states. By Definition 15, livelock-freeness implies that every other marking in $RG(\mathcal{N})$ reaches some of such final marking, thus getting encoded by $Chain(A)$ into a transient state. $\triangleleft$

5.3. Outcome probability and stochastic languages for livelock-free bounded LSPs

Lemma 1 provides the basis for showing that the outcome probability problem can be casted as an absorption probability problem, and hence analytically solved through the technique mentioned in [Theorem 1](#).

Theorem 2. *For every livelock-free bounded LSP $\mathcal{N}$ and set $F \subseteq \mathcal{N}.M_f$, we have that $\text{OUTCOME-PROB}(\mathcal{N}, F)$ has a unique solution, which coincides with that of $\text{ABSORPTION-PROB}(\text{Chain}(\mathcal{N}), \mathcal{N}.s_0, F)$.* $\triangleleft$

Proof. [Lemma 1](#) guarantees that $\text{Chain}(\mathcal{N})$ is an absorbing Markov chain. This implies, by [Theorem 1](#), that $\text{ABSORPTION-PROB}(\text{Chain}(\mathcal{N}), \mathcal{N}.s_0, F)$ has a unique solution. By contrasting the definition of path through a LSP as per [Definition 9](#), with the definition of proper path as per [Definition 22](#), we have that every path through $\mathcal{N}$ corresponds one-to-one to a proper walk over $\text{Chain}(\mathcal{N})$ from s_0 . Hence:

$$\begin{aligned} \text{OUTCOME-PROB}(\mathcal{N}, F) &= \sum_{\eta \text{ path through } \mathcal{N} \text{ ending in some } m \in F} \mathbb{P}_{\mathcal{N}}(\eta) \\ &= \sum_{w \text{ proper walk of } \text{Chain}(\mathcal{N}) \text{ from } s_0 \text{ to some } m \in F} \mathbb{P}_{\text{Chain}(\mathcal{N})}(w) \\ &= \text{ABSORPTION-PROB}(\text{Chain}(\mathcal{N}), \mathcal{N}.s_0, F) \quad \triangleleft \end{aligned}$$

Considering [Theorem 2](#), the encoding of livelock-free bounded LSPs into absorbing Markov chains, and the definition of the corresponding systems of equations, we can directly solve the outcome probability problem $\text{OUTCOME-PROB}(\mathcal{N}, F)$ for a livelock-free bounded LSP and a nonempty set $F \subseteq \mathcal{N}.M_f$ by extracting the value of variable $x_{\mathcal{N}.m_0}$ from the unique solution of the following system $\mathcal{E}_{\mathcal{N}}^{M_f}$ of equations:

$$x_{m_i} = 1 \quad \text{for each marking } m_i \in F \quad (4)$$

$$x_{m_j} = 0 \quad \text{for each marking } m_j \in \mathcal{N}.M_f \setminus F \quad (5)$$

$$x_{m_k} = \sum_{\langle m_k, l, m'_k \rangle \in \text{succ}_{\text{Chain}(\mathcal{N})}(m_k)} p(\langle m_k, l, m'_k \rangle) \cdot x_{m'_k} \quad \text{for each marking } m_k \in \text{RG}(\mathcal{N}).S \setminus M_f \quad (6)$$

Example 10. Consider the bounded LSP $\mathcal{N}_{\text{order}}$ ([Fig. 3](#)). We want to solve $\text{OUTCOME-PROB}(\mathcal{N}_{\text{order}}, m_h)$ to compute the probability that a created order is eventually paid and shipped. To do so, we encode the reachability graph of [Fig. 5](#) into the system $\mathcal{E}_{\mathcal{N}_{\text{order}}}^{m_h}$. Specifically, noticing that the overall sum of all probability values appearing in the outgoing edges of a given marking is always 1, we get:

$$\begin{aligned} x_{m_h} &= 1 \\ x_{m_c} &= x_{m_r} = x_{m_5} = 0 \\ x_{m_{10}} &= x_{m_{11}} = x_{m_{12}} = x_{m_h} = 1 \\ x_{m_8} &= x_{m_9} = \rho_e x_{m_{10}} + \rho_l x_{m_{11}} = \rho_e + \rho_l = 1 \\ x_{m_6} &= x_{m_7} = \rho_{i_3} x_{m_3} + \rho_p x_{m_8} + \rho_{c_2} x_{m_c} = \rho_{i_3} x_{m_3} + \rho_p \\ x_{m_4} &= \rho_{s_3} x_{m_5} + \rho_{s_4} x_{m_6} = \rho_{s_4} \rho_{i_3} x_{m_3} + \rho_{s_4} \rho_p \\ x_{m_3} &= \rho_{i_2} x_{m_2} + \rho_f x_{m_4} + \rho_{c_1} x_{m_c} = \rho_{i_2} x_{m_2} + \rho_f \rho_{s_4} \rho_{i_3} x_{m_3} + \rho_f \rho_{s_4} \rho_p \\ &= \frac{\rho_{i_2}}{1 - \rho_f \rho_{s_4} \rho_{i_3}} x_{m_2} + \frac{\rho_f \rho_{s_4} \rho_p}{1 - \rho_f \rho_{s_4} \rho_{i_3}} \\ x_{m_2} &= \rho_{s_1} x_{m_1} + \rho_{s_2} x_{m_3} = \rho_{s_1} x_{m_1} + \frac{\rho_{s_2} \rho_{i_2}}{1 - \rho_f \rho_{s_4} \rho_{i_3}} x_{m_2} + \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_p}{1 - \rho_f \rho_{s_4} \rho_{i_3}} \\ &= \frac{\rho_{s_1} - \rho_{s_1} \rho_f \rho_{s_4} \rho_{i_3}}{1 - \rho_f \rho_{s_4} \rho_{i_3} - \rho_{s_2} \rho_{i_2}} x_{m_1} + \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_p}{1 - \rho_f \rho_{s_4} \rho_{i_3} - \rho_{s_2} \rho_{i_2}} \\ x_{m_0} &= x_{m_1} = x_{m_2} = \frac{\rho_{s_1} - \rho_{s_1} \rho_f \rho_{s_4} \rho_{i_3}}{1 - \rho_f \rho_{s_4} \rho_{i_3} - \rho_{s_2} \rho_{i_2}} x_{m_0} + \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_p}{1 - \rho_f \rho_{s_4} \rho_{i_3} - \rho_{s_2} \rho_{i_2}} \\ &= \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_p}{1 - \rho_{s_1} - \rho_{s_2} \rho_{i_2} - \rho_f \rho_{s_4} \rho_{i_3} + \rho_{s_1} \rho_f \rho_{s_4} \rho_{i_3}} \end{aligned}$$

$$\begin{aligned} &= \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_p}{1 - \rho_{s_1} - \rho_{s_2} \rho_{i_2} - \rho_f \rho_{s_4} \rho_{i_3} (1 - \rho_{s_1})} \\ &= \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_p}{1 - \rho_{s_1} - \rho_{s_2} \rho_{i_2} - \rho_{s_2} \rho_f \rho_{s_4} \rho_{i_3}} \end{aligned}$$

If we assume that the weights of $\mathcal{N}_{\text{order}}$ are all equal, the probability distributions for choosing the next transition are all uniform, so that $\rho_{s_1} = \rho_{s_2} = \frac{1}{2}$, $\rho_{i_2} = \rho_f = \rho_{c_1} = \frac{1}{3}$, $\rho_{s_3} = \rho_{s_4} = \frac{1}{2}$, $\rho_{i_3} = \rho_p = \rho_{c_2} = \frac{1}{3}$, and $\rho_e = \rho_l = \frac{1}{2}$. Under this assumption, we thus get $x_{m_0} = \frac{1/36}{1 - 1/2 - 1/6 - 1/36} = \frac{1/36}{11/36} = \frac{1}{11} \approx 0.09$.

With an analogous approach, the probability that an order gets cancelled is $\frac{4}{11}$, and the one that an order gets rejected is $\frac{6}{11}$. $\triangleleft$

As the example shows, the total sum of probabilities associated to reaching the whole set of final markings is, as expected, 1: every order that finishes does so in one of the categories of completion (i.e., one of the final markings). This actually generalises to any livelock-free bounded LSP: the outcome probability calculated by considering the whole set of final markings corresponds to 1.

Theorem 3. *For every livelock-free bounded LSP $\mathcal{N}$, we have that $\text{OUTCOME-PROB}(\mathcal{N}, \mathcal{N}.M_f) = 1$.* $\triangleleft$

Proof. The claim is a direct consequence of [Theorem 2](#), and the fact that the probability of reaching an absorbing state in an absorbing Markov chain is 1 - as proved in [[24](#), Theorem 11.3]. $\triangleleft$

The effect of [Theorem 3](#) is that the trace language of a livelock-free bounded LSP is indeed a stochastic language.

Definition 24 (Stochastic Language). A stochastic language over an alphabet L of symbols is a total function $\Psi : L^* \rightarrow [0, 1]$ mapping each sequence over L onto a probability value, so that $\sum_{\xi \in L^*} \Psi(\xi) = 1$. $\triangleleft$

Definition 25 (Induced Stochastic Language). The induced stochastic language of a livelock-free bounded LSP $\mathcal{N}$ is the function $\mathcal{P}_{\mathcal{N}} : \mathcal{A}^* \rightarrow [0, 1]$ defined as follows: for every $\sigma \in \mathcal{A}^*$, we have

$$\mathcal{P}_{\mathcal{N}}(\sigma) = \begin{cases} \mathbb{P}_{\mathcal{N}}(\sigma) & \text{if } \sigma \text{ is a model trace of } \mathcal{N} \\ 0 & \text{otherwise} \end{cases}$$

Corollary 1. *Let $\mathcal{N}$ be a livelock-free bounded LSP. Then $\mathcal{P}_{\mathcal{N}}$ is indeed a stochastic language in the sense of [Definition 24](#).* $\triangleleft$

5.4. Dealing with livelocks

We now show how the technique introduced in [Section 5.3](#) for livelock-free bounded LSPs can be generalised to arbitrary bounded LSPs with minor adaptations: the crux is to deal with livelocks by recognising that they are not able to reach any final marking, and hence their states should be assigned to a probability value of 0.

This problem is well-known in Markov chain analysis. In fact, the same approach used to solve the absorption probability problem can be adopted, in spirit, to deal with arbitrary Markov chains, in particular to compute the probability that a walk from a state v_i culminates in another state v_f . However, as the next example shows, in case of livelocks the system of linear equations recalled before becomes indeterminate.

Example 11. Consider the bounded LSP $\mathcal{N}_{\text{live}}$ from [Example 7](#) and [Fig. 6](#). If we try to compute the outcome probability of its two final markings $[q_1]$ and $[q_5]$ by applying verbatim the approach used so far, we get the following system of linear equations, directly derived from the reachability graph of [Fig. 6\(b\)](#):

$$x_{m_5} = 1 \quad x_{m_2} = x_{m_3}$$

$$\begin{aligned} x_{m_4} &= x_{m_5} & x_{m_1} &= 1 \\ x_{m_3} &= \rho_d x_{m_2} + \rho_e x_{m_3} & x_{m_0} &= \rho_a x_{m_1} + \rho_b x_{m_2} + \rho_f x_{m_4} \end{aligned}$$

By recalling that, by definition, $\rho_d + \rho_e = 1$, the system yields $x_{m_3} = \rho_d x_{m_3} + \rho_e x_{m_3} = (\rho_d + \rho_e) x_{m_3} = x_{m_3}$, and so is indeterminate. This is caused by the livelock markings $[q_2]$ and $[q_3]$, which induce a cycle from which none of the target markings can be reached. The minimal non-negative solution of the system then assigns $x_{m_3} = x_{m_2} = 0$ and, in turn, $x_{m_0} = \rho_a \cdot \triangleleft$

The indeterminacy shown in [Example 11](#) is solved in Markov chain analysis by turning the system of equalities into an optimisation problem, where one is interested in extracting the minimum non-negative solution [\[25,34\]](#). This corresponds to assigning 0 to each state variable that is causing the indeterminacy. In the case of bounded LSPs, such state variables correspond to livelock markings. To deal with arbitrary bounded LSPs without resorting to an optimisation problem, we then proceed as follows. The outcome probability problem $\text{OUTCOME-PROB}(\mathcal{N}, F)$ for an arbitrary bounded LSP and a nonempty set $F \subseteq \mathcal{N}.M_f$ is solved by extracting the value of variable $x_{\mathcal{N}.m_0}$ from the unique solution of $\mathcal{E}_{\mathcal{N}}^{M_f}$, now generalised as follows:

$$x_{m_i} = 1 \quad \text{for each marking } m_i \in F \quad (7)$$

$$x_{m_j} = 0 \quad \text{for each marking } m_j \in \mathcal{N}.M_f \setminus F \quad (8)$$

$$x_{m_i} = 0 \quad \text{for each livelock marking } m_i \in \text{RG}(\mathcal{N}).S \quad (9)$$

$$x_{m_k} = \sum_{\langle m_k, l, m'_k \rangle \in \text{succ}_{\text{RG}(\mathcal{N})}(m_k)} p(\langle m_k, l, m'_k \rangle) \cdot x_{m'_k} \quad \text{for each other marking } m_k \in \text{RG}(\mathcal{N}).S \quad (10)$$

Recall that detecting livelock markings can be done over the reachability graph of the input bounded LSP via simple graph (non-)reachability checks. Such checks do not involve probabilities at all.

Example 12. We revise [Example 11](#) by solving $\text{OUTCOME-PROB}(\mathcal{N}_{\text{live}}, \{[q_1], [q_5]\})$ as follows. The system $\mathcal{E}_{\mathcal{N}_{\text{live}}}^{\{[q_1], [q_5]\}}$ corresponds to:

$$\begin{aligned} x_{m_5} &= x_{m_1} = 1 & x_{m_2} &= x_{m_3} = 0 \\ x_{m_4} &= x_{m_5} & x_{m_0} &= \rho_a x_{m_1} + \rho_b x_{m_2} + \rho_f x_{m_4} \end{aligned}$$

Hence, we get $x_{m_0} = \rho_a + \rho_f \cdot \triangleleft$

As witnessed by [Example 12](#), in case of bounded LSPs with livelocks, the outcome probability value p computed over all final markings is not 1, as instead it is guaranteed for livelock-free bounded LSPs (cf. [Theorem 3](#)). This also means that every bounded LSP with livelocks does not induce a stochastic language over its traces, but only a form of “incomplete” stochastic language, where the complement probability value $1 - p$ indicates the probability that the process gets stuck in a livelock.

Remark 2. For a bounded LSP $\mathcal{N}$ containing livelocks, we get the following:

- $\text{OUTCOME-PROB}(\mathcal{N}, \mathcal{N}.M_f) < 1$;
- $1 - \text{OUTCOME-PROB}(\mathcal{N}, \mathcal{N}.M_f)$ is the probability that an execution starting from $\mathcal{N}.m_0$ ends up into a livelock. $\triangleleft$

Example 13. By continuing on [Example 12](#), we have that the probability that an execution of $\mathcal{N}_{\text{live}}$ starting from $\mathcal{N}_{\text{live}}.m_0 = [q_0]$ ends up into a livelock is $1 - \text{OUTCOME-PROB}(\mathcal{N}_{\text{live}}, \{[q_1], [q_5]\}) = 1 - \rho_a - \rho_f = \rho_b$, which is in fact the probability of firing t_{02} from the initial marking. $\triangleleft$

6. Computing specification and trace probabilities

We now further leverage the connection between bounded LSPs and discrete-time Markov chains (cf. [Definition 23](#)), to deal with the computation of **specification probabilities**. A specification is a compact description of (possibly infinitely many) traces of interest, and the problem is about measuring what is the probability that a trace produced by the bounded LSP of interest indeed belongs to the specification. This problem is also known under the name of qualitative model checking [\[25, Ch. 10\]](#), to stress that the specification is *qualitative*, i.e., non-probabilistic.

To solve the problem, we borrow the main idea from [\[25, Ch. 10\]](#) that underlies the construction of the cross-product stochastic transition system that describes all and only the behaviour that belong to both the specification and the bounded LSP under scrutiny. However, we have to adapt this construction to reflect two distinctive features of our setting:

- traces have a finite length, and this has to be taken into account in the choice of formalism for describing the specification, as well as when constructing the cross-product;
- the behaviour described by the specification are traces, while those of the bounded LSP are paths over transitions that also include silent steps, a mismatch that has to be resolved before constructing the cross product.

We proceed by fixing and justifying the formalism we use to represent specifications. We then formalise the problem of specification probability, and describe how to solve it in three steps: (i) infusing silent steps in the specification; (ii) computing the cross-product of the altered specification and the reachability graph of the bounded LSP; and (iii) solving an outcome probability problem on the cross product.

Finally, we show that the solution of the specification probability problem yields, as a by-product, the solution of the **trace probability** problem (as described in [Definition 20](#)).

6.1. Automata-based specifications

As already pointed out, a specification intentionally describes a (possibly infinite) set of desired finite-length traces. Such traces are defined over the task names in $\mathcal{A}$ (without τ). As a natural formalism to capture such specifications, we employ (deterministic) finite-state automata. To avoid confusion between the sequences accepted by automata and paths/traces of LSPs, we use the standard term of *words* for automata.

Definition 26 (DFA). A *deterministic finite-state automaton* (DFA) over L is a tuple $A = \langle L, S, s_0, S_f, \delta \rangle$, where:

- L is a finite *alphabet* of symbols;
- S is a finite set of *states*, with $s_0 \in S$ the *initial state* and $S_f \subseteq S$ the set of *final states*;
- $\delta : S \times L \rightarrow S$ is a transition function that, given a state $s \in S$ and a label $l \in L$, returns the successor state $\delta(s, l)$. $\triangleleft$

Definition 27 (Word, Acceptance, Language). Given a finite alphabet L , a word over L is a finite sequence of symbols from L^* . A DFA A over L accepts a word $\sigma = l_0, \dots, l_n$ if there exists a sequence of states $s_0, \dots, s_{n+1}$ such that:

- $s_0 = A.s_0$;
- $s_{n+1} \in S_f$;
- for every $i \in \{0, \dots, n\}$, we have $s_{i+1} = \delta(s_i, l_i)$.

The *language* $\mathcal{L}(A)$ of A is the set of all words accepted by A . $\triangleleft$

Remark 3. Dealing with DFAs in turn provides an operational solution for dealing with a wide range of formalisms to specify traces:

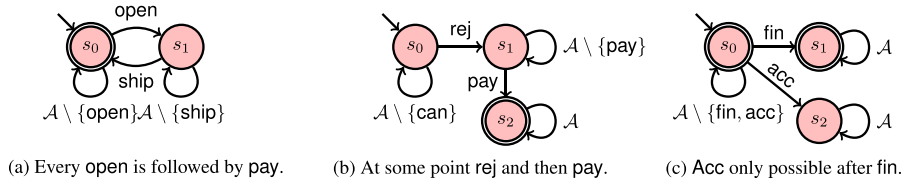


Fig. 7. DFAs of three properties for the order-to-cash example. A single edge labelled by a set $\mathcal{A}$ of task names compactly describes a set of edges, each labelled by a task name from $\mathcal{A}$.

- It expresses all non-deterministic automata (NFAs), as each NFA can be encoded into a corresponding DFA [35].
- For the same reason [35], it expresses all regular expressions.
- It expresses all declarative specifications formalised in Linear Temporal Logic over finite traces (LTL_f) or the richer Linear Dynamic Logic over finite traces (LDL_f), as such logics have an automata-theoretic characterisation based on NFAs/DFAs [36, 37].
- In turn, it expresses all specifications captured in Declare [38, 39], possibly extended with meta-constraints [37]. $\triangleleft$

Example 14. Fig. 7 shows three specifications of interest for our running example. Specifically:

- The DFA in Fig. 7(a) captures those words where every time an order is opened, it is later shipped (possibly executing other tasks in between).
- The DFA in Fig. 7(b) captures those words where the order is eventually rejected and later paid.
- The DFA in Fig. 7(c) captures those words where if an order is accepted, then it has been previously finalised. $\triangleleft$

6.2. Defining and solving the specification probability problem

Noticing that words over $\mathcal{A}$ correspond to traces of LSPs, we are now ready to define the **specification probability** problem.

Problem 3 (SPEC-PROB($\mathcal{N}$, A)).

Input: bounded LSP $\mathcal{N}$, DFA A over $\mathcal{A}$;

Output: probability $\mathbb{P}_{\mathcal{N}}(A) = \sum_{\sigma \text{ path through } \mathcal{N} \text{ s.t. } \sigma \in \mathcal{L}(A)} \mathbb{P}_{\mathcal{N}}(\sigma)$. $\triangleleft$

To solve the problem, we need to account for three different aspects:

1. deal with the mismatch between paths over $\mathcal{N}$ and words accepted by A ;
2. single out all and only those paths through $\mathcal{N}$ that are also words accepted by A ;
3. compute the collective probability of all such traces.

We tackle these three aspects with three corresponding steps.

Silenced DFA. Given a specification DFA describing desired words over $\mathcal{A}$, we transform it into a corresponding DFA describing corresponding sequences of labels over Σ . The idea is to make the resulting DFA continue to describe the same desired traces, but accepts padding their visible steps with an arbitrary number of silent steps in between.

Definition 28 (Silenced DFA). Given a DFA $A = \langle \mathcal{A}, S, s_0, S_f, \delta \rangle$ over $\mathcal{A}$, its *silenced DFA* $\bar{A}$ is the DFA $\langle \Sigma, S', s'_0, S'_f, \delta' \rangle$ over Σ satisfying the following conditions:

- states (including initial and final ones) are identical to A , that is, $S' = S$, $s'_0 = s_0$, and $S'_f = S_f$;
- δ' expands δ by introducing a τ -labelled self-loop in each state, that is, $\delta' = \delta \cup \{ \langle s, \tau \rangle \rightarrow s \mid s \in S \}$. $\triangleleft$

Fig. 8 shows the silenced DFAs corresponding to the DFAs in Fig. 7.

We now establish a key correspondence between DFAs and their silenced versions, when characterising traces of an LSP.

Lemma 2. Let $\mathcal{N}$ be an LSP, and A be a DFA over $\mathcal{A}$. For every model trace σ of $\mathcal{N}$ and for every path η of $\mathcal{N}$ inducing σ , we have that σ is accepted by A if and only if the sequence of labels $\mathcal{N}.N.\ell(\eta)$ is accepted by $\bar{A}$. $\triangleleft$

Proof. The claim directly follows from Definitions 10 and 28, noticing that the difference between a trace σ induced by a path η and the sequence of labels $l = \mathcal{N}.N.\ell(\eta)$ is that l may insert arbitrarily many τ symbols at the beginning and of σ , as well as between every consecutive tasks appearing therein. $\triangleleft$

Product stochastic transition system. Given a bounded LSP $\mathcal{N}$ and a DFA B over Σ , we use $RG(\mathcal{N})$ and B to construct a *product* stochastic transition system generates all and only those paths through $\mathcal{N}$ whose label sequences are accepted by B , retaining the probabilities of those paths through $\mathcal{N}$. This can be done by the standard product automaton construction, with the only difference that we need to retain the stochastic information coming from $\mathcal{N}$. We then use this as a basis to solve the specification probability problem, relying on silenced-DFAs.

Definition 29 (Product System). Let $\mathcal{N}$ be a bounded LSP with $RG(\mathcal{N}) = \langle S^1, s_0^1, S_f^1, \rho^1, p^1 \rangle$, and $B = \langle \Sigma^2, S^2, s_0^2, S_f^2, \delta^2 \rangle$ a DFA over Σ . The *product system* $Y_{\mathcal{N}}^B$ of $\mathcal{N}$ and B is a stochastic transition system $\langle S, s_0, S_f, \rho, p \rangle$ whose states are pairs of states from $S^1 \times S^2$, and whose components are defined by mutual induction as the minimal sets satisfying the following conditions:

1. $s_0 = \langle s_0^1, s_0^2 \rangle$, $s_0 \in S$;
2. for every state $\langle s^1, s^2 \rangle \in S$ and every label $l \in \Sigma^2$ such that
 - (i) $\langle s^1, t^1, s^{1'} \rangle \in \rho^1$ for some $s^{1'} \in S^1$ and transition $t^1 \in \mathcal{N}.N.T$ with $\mathcal{N}.N.\ell(t^1) = l$, and
 - (ii) $\delta^2(s^2, l) = s^{2'}$ for some $s^{2'} \in S^2$,

and taking $s' = \langle s^{1'}, s^{2'} \rangle$ we have:

- (a) $s' \in S$,
- (b) $\langle s, t^1, s' \rangle \in \rho$,
- (c) $p(\langle s, t^1, s' \rangle) = p_1(\langle s_1, t^1, s_1' \rangle)$,
- (d) if $s^{1'} \in S_f^1$ and $s^{2'} \in S_f^2$, then $s' \in S_f$. $\triangleleft$

Remark 4. The product system as per Definition 29 is not a *complete* stochastic transition system: there may be states whose successor probabilities do not add up to one. However, it can be made complete as follows:

- introduce a fresh, non-final sink state s_{sink} , and introduce a new net transition t_{sink} that is not used in the input LSP;
- for every state s in the product system such that the sum of the probabilities of its outgoing edges correspond to $p < 1$, add an edge between s and s_{sink} , labelling it with t_{sink} and assigning probability $1 - p$ to it.

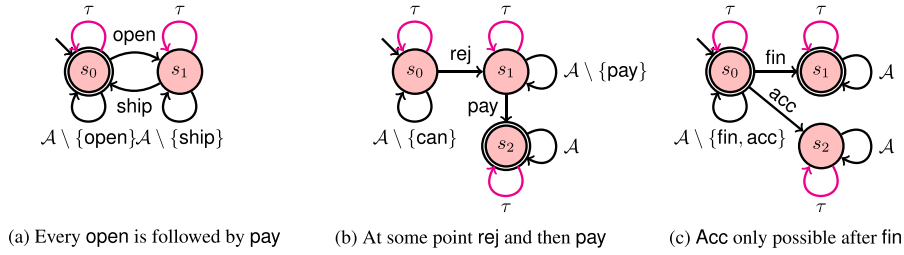


Fig. 8. Silenced DFAs corresponding to the three DFAs in Fig. 7.

As we will see next, we are going to use the (Markov chain underlying the) product system for calculating probabilities. Hence, this completion can be avoided (as state s_{sink} would get assigned a probability of 0). $\triangleleft$

Specification probability as outcome probability. We are now ready to bring everything together, exploiting the notions of silenced DFAs and product systems to show how the SPEC-PROB problem can be reduced to OUTCOME-PROB, invoked on $Y_{\mathcal{N}}^{\bar{A}}$, considering all its final states.

Theorem 4. For every bounded LSP $\mathcal{N}$ and DFA A over $\mathcal{A}$, we have that $\text{SPEC-PROB}(\mathcal{N}, A) = \text{OUTCOME-PROB}(Y_{\mathcal{N}}^{\bar{A}}, Y_{\mathcal{N}}^{\bar{A}} \cdot S_f)$. $\triangleleft$

Proof. First of all, we recall the definition of SPEC-PROB, considering that the probability of a model trace of $\mathcal{N}$ is the sum of the probabilities of the paths through $\mathcal{N}$ inducing that trace: $\text{SPEC-PROB}(\mathcal{N}, A) = \sum_{\sigma \text{ model trace of } \mathcal{N} \text{ s.t. } \sigma \in \mathcal{L}(A)} \mathbb{P}_{\mathcal{N}}(\sigma) = \sum_{\eta \text{ path through } \mathcal{N} \text{ inducing } \sigma \text{ s.t. } \sigma \in \mathcal{L}(A)} \mathbb{P}_{\mathcal{N}}(\eta)$. By Lemma 2, we have that the acceptance of a trace by A can be lifted to acceptance of the label sequence induced by its corresponding paths over $\bar{A}$. This yields: $\text{SPEC-PROB}(\mathcal{N}, A) = \sum_{\eta \text{ path through } \mathcal{N} \text{ s.t. } \mathcal{N}.N.\mathcal{L}(\eta) \in \mathcal{L}(A)} \mathbb{P}_{\mathcal{N}}(\eta)$. By Definition 29, the paths mentioned in this sum are precisely those of $Y_{\mathcal{N}}^{\bar{A}}$. By the same definition, we also know that the probabilities of such paths as assigned by $Y_{\mathcal{N}}^{\bar{A}}$ are the same as those assigned by $\mathcal{N}$. Hence: $\text{SPEC-PROB}(\mathcal{N}, A) = \sum_{\eta \text{ path through } Y_{\mathcal{N}}^{\bar{A}}} \mathbb{P}_{Y_{\mathcal{N}}^{\bar{A}}}(\eta) = \text{OUTCOME-PROB}(Y_{\mathcal{N}}^{\bar{A}}, Y_{\mathcal{N}}^{\bar{A}} \cdot S_f)$. $\triangleleft$

6.3. Computing trace probabilities

As already pointed out, a key problem in stochastic conformance checking [15,17] is that of computing the probability of a trace in a stochastic setting. Starting from Definition 20, we cast such a problem in the context of bounded LSPs as follows:

Problem 4 ($\text{TRACE-PROB}(\mathcal{N}, \sigma)$).

Input: Bounded LSP $\mathcal{N}$, trace σ over $\mathcal{A}$;

Output: Probability $\mathbb{P}_{\mathcal{N}}(\sigma)$. $\triangleleft$

To solve this problem, we straightforwardly encode a trace σ into a corresponding DFA over $\mathcal{A}$ that accepts exactly σ . We call this the *trace DFA* of σ :

Definition 30 (*Trace DFA*). Given a trace $\sigma = a_0, \dots, a_n$ over $\mathcal{A}$, its *trace DFA* A_{σ} is the DFA $\langle \mathcal{A}, S, s_0, S_f, \delta \rangle$ over $\mathcal{A}$ such that:

1. $S = \{s_0, \dots, s_{n+1}\}$ contains $n+1$ states;
2. $S_f = \{s_{n+1}\}$;
3. for every $i \in \{0, \dots, n\}$, $\delta(s_i, a_i) = s_{i+1}$ (and nothing else is in δ). $\triangleleft$

This immediately yields the following key result:

Theorem 5. For every bounded LSP $\mathcal{N}$ and every trace σ over $\mathcal{A}^*$, we have that $\text{TRACE-PROB}(\mathcal{N}, \sigma) = \text{SPEC-PROB}(\mathcal{N}, A_{\sigma})$. $\triangleleft$

Proof. Direct from the definitions of the problems, noticing that $\mathcal{L}(A_{\sigma}) = \{\sigma\}$. $\triangleleft$

Example 15. Consider the trace $\sigma_{ar} = \text{open}, \text{fin}, \text{acc}, \text{fin}, \text{rej}$ from Example 5. We now compute the probability that $\mathcal{N}_{\text{order}}$ generates trace σ_{ar} , where an order is filled, finalised, accepted, then modified, finalised again, and this second time rejected. Following the described technique, we first transform σ_{ar} into its trace DFA $A_{\sigma_{ar}}$, and then further into its silenced trace DFA $\bar{A}_{\sigma_{ar}}$. This is shown in Fig. 9(a) and (b). We then compute the product system $Y_{RG(\mathcal{N}_{\text{order}})}^{\bar{A}_{\sigma_{ar}}}$ of $\bar{A}_{\sigma_{ar}}$ and $RG(\mathcal{N}_{\text{order}})$ (from Fig. 5), obtaining the (incomplete, cf. Remark 4) stochastic transition system of Fig. 9(b) - notice how silent transitions unfold in this transition system. Finally, we construct $\mathcal{E}^{\{(7.5)\}}_{Y_{RG(\mathcal{N}_{\text{order}})}^{\bar{A}_{\sigma_{ar}}}}$. Recalling that

by definition $\rho_{s_1} + \rho_{s_2} = 1$, we then get:

$$\begin{aligned}
 x_{55} &= 1 \\
 x_{44} &= \rho_{s_3} x_{54} = \rho_{s_3} x_{55} = \rho_{s_3} \\
 x_{23} &= \rho_{s_1} x_{13} + \rho_{s_2} x_{33} = \rho_{s_1} x_{23} + \rho_{s_2} x_{33} = \frac{\rho_{s_2}}{1 - \rho_{s_1}} x_{33} \\
 x_{33} &= \rho_f x_{44} + \rho_{i_2} x_{23} = \rho_f \rho_{s_3} + \frac{\rho_{i_2} \rho_{s_2}}{1 - \rho_{s_1}} x_{33} = \frac{\rho_f \rho_{s_3}}{1 - \rho_{i_2}} \\
 x_{42} &= \rho_{s_4} x_{62} = \rho_{s_4} x_{73} = \rho_{s_4} \rho_{i_3} x_{33} = \frac{\rho_{s_4} \rho_{i_3} \rho_f \rho_{s_3}}{1 - \rho_{i_2}} \\
 x_{31} &= \rho_{i_2} x_{21} + \rho_f x_{42} = \rho_{i_2} x_{21} + \frac{\rho_f \rho_{s_4} \rho_{i_3} \rho_f \rho_{s_3}}{1 - \rho_{i_2}} \\
 x_{21} &= \rho_{s_1} x_{11} + \rho_{s_2} x_{31} = \rho_{s_1} x_{11} + \rho_{s_2} \rho_{i_2} x_{21} + \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_{i_3} \rho_f \rho_{s_3}}{1 - \rho_{i_2}} \\
 &= \frac{\rho_{s_1}}{1 - \rho_{s_2} \rho_{i_2}} x_{11} + \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_{i_3} \rho_f \rho_{s_3}}{(1 - \rho_{i_2})(1 - \rho_{s_2} \rho_{i_2})} \\
 x_{00} &= x_{11} = x_{21} = \frac{\rho_{s_1}}{1 - \rho_{s_2} \rho_{i_2}} x_{00} + \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_{i_3} \rho_f \rho_{s_3}}{(1 - \rho_{i_2})(1 - \rho_{s_2} \rho_{i_2})} \\
 &= \frac{\rho_{s_2} \rho_f \rho_{s_4} \rho_{i_3} \rho_f \rho_{s_3}}{(1 - \rho_{i_2})(1 - \rho_{s_2} \rho_{i_2})} \frac{1 - \rho_{s_2} \rho_{i_2}}{\rho_{s_2} (1 - \rho_{i_2})} = \frac{\rho_f \rho_{s_4} \rho_{i_3} \rho_f \rho_{s_3}}{(1 - \rho_{i_2})^2}
 \end{aligned}$$

which yields the solution to the $\text{TRACE-PROB}(\mathcal{N}_{\text{order}}, \sigma)$ problem. If all transition weights have the same value (as in Example 10), we get as result probability $\frac{1/3 \cdot 1/2 \cdot 1/3 \cdot 1/3 \cdot 1/2}{(1-1/3)^2} = \frac{1}{48} \sim 0.02$. $\triangleleft$

We close with an important observation: a variant of the specification probability technique can be used to realise a form of *white-box outcome prediction*, to compute the probability of a given outcome given a partial, observed trace σ that describes an ongoing process execution. This is done through SPEC-PROB, making sure that the set of final markings of the input net coincides with the outcome of interest, and that the input DFA encodes the language corresponding to all the traces that have σ as prefix. The latter can be done by a minor modification of the trace DFA of σ , where the final state has a self-loop for every element of $\mathcal{A}$. This technique can be used, e.g., to show that in our running example, as soon as a trace contains a payment, then the final outcome consisting of the happy completion state is expected to be reached with probability 1.

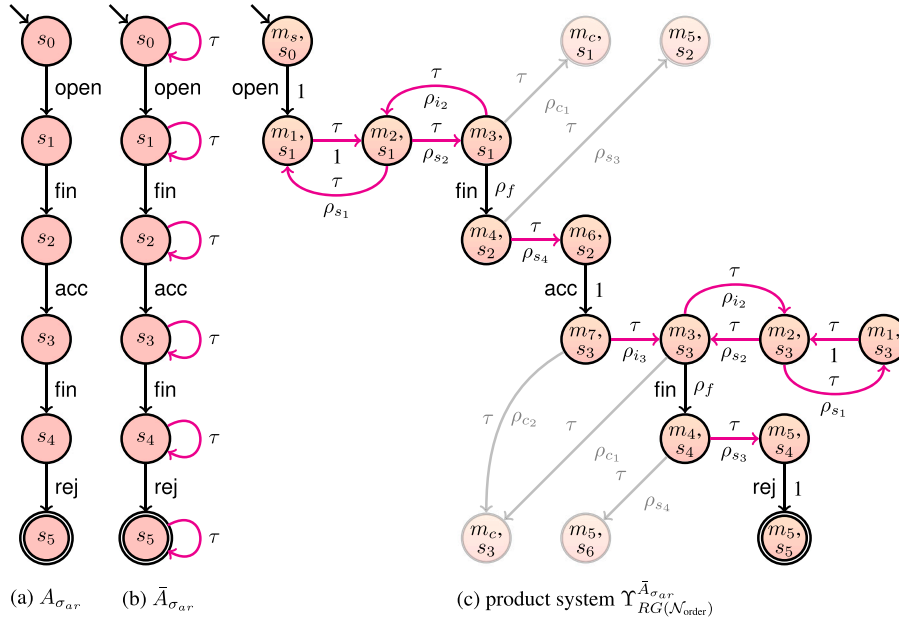


Fig. 9. (a) Trace DFA and (b) silenced trace DFA for trace $\sigma = \text{open, fin, acc, fin, rej}$, and (c) product system with the reachability graph of Fig. 5; the portion of the transition system that cannot reach the final state is greyed out.

7. Stochastic compliance with probabilistic declare

We now employ the verification machinery from Section 6.2 to deal with probabilistic temporal properties. In particular, we introduce *stochastic compliance*, where the stochastic behaviour induced by a bounded LSP is contrasted with a set of probabilistic temporal constraints, each expressing a qualitative specification paired with a condition on the expected probability that the specification is indeed satisfied. Since multiple probabilistic constraints can be given, combined conditions on the probability of their satisfaction/violation must be obtained. To systematically handle this, we rely on the ProbDeclare language from [40], which extends the well-known Declare language [38,39] with constraint probabilities. Before defining and solving our stochastic conformance problem, we briefly recall ProbDeclare.

7.1. Probabilistic declare

Declare is a constraint-based declarative language for modelling flexible processes [38,39]. The acceptable behaviours supported by a process are implicitly characterised using LTL_f constraints: a Declare specification consists of a set of LTL_f constraints, which must be all respected in a complete process execution. This comes with a *crisp* interpretation of constraints: a complete trace satisfies a Declare specification if it satisfies *every* constraints contained therein. For an overall introduction of Declare and corresponding key analysis and process mining tasks, the interested reader is referred to [41].

By exploiting a probabilistic version of LTL_f as underlying temporal logic [42], the crisp semantics of Declare has been lifted to an uncertain one in [40], opening up a full range of new tasks for (probabilistic) declarative process mining [18]. In the resulting ProbDeclare framework, each constraint comes with a probability condition identifying a set of probabilities for which a trace generated by the process satisfies that constraint. More precisely, the semantics is based on stochastic languages, which assign a probability mass to each trace in a (possibly infinite) set, as recalled in Definition 24. Specifically, a stochastic language satisfies a probabilistic constraint if, considering only those traces in the language that satisfy the constraint, their overall probability mass satisfies the probability condition attached

to the constraint. This makes ProbDeclare a natural candidate to express probabilistic temporal properties over livelock-free bounded LSPs, whose model traces with their probabilities indeed capture stochastic languages (cf. Corollary 1).

In ProbDeclare, probabilities are assigned to traces, compatibly with the semantics of bounded LSPs. On the other hand, as recalled in Remark 3, the task of computing the probability that a bounded LSP meets a specification provided with a DFA can be seamlessly used to capture the case where the specification is given in LTL_f . Hence, we do not need to introduce LTL_f in the context of the current article — we simply rely on informal presentation of constraints, formalising them directly using DFAs. For a thorough introduction to LTL_f and its automata-theoretic characterisation, see [36,37].

The main question for ProbDeclare is how to characterise which stochastic languages satisfy a set of probabilistic constraints, considering how the temporal and probability components of each constraint interact with those of the others. We recall from [18] the necessary definitions to obtain such a characterisation.

Definition 31 (Probabilistic Constraint). A *probabilistic constraint* over $\mathcal{A}$ is a triple $\langle \varphi, \bowtie, p \rangle$, where: (i) φ is an LTL_f formula over $\mathcal{A}$ representing the *constraint formula*; (ii) $\bowtie \in \{=, \neq, \leq, <, >\}$ is the *constraint probability operator*; (iii) p is a rational value in $[0, 1]$ representing the *constraint probability*. $\triangleleft$

Definition 32 (ProbDeclare Specification). A *ProbDeclare specification* is a set D of probabilistic constraints over $\mathcal{A}$. $\triangleleft$

Differently from [18,40], we do not single out crisp constraints from genuinely probabilistic constraints. This is just for simplicity of presentation, recalling that a crisp constraint φ can be represented as the corresponding probabilistic constraint $\langle \varphi, =, 1 \rangle$.

Definition 33 (ProbDeclare Satisfaction Relation). A stochastic language Ψ satisfies a ProbDeclare specification D if for every probabilistic constraint $\langle \varphi, \bowtie, p \rangle \in D$, we have that:

$$\sum_{\sigma \in \mathcal{A}^* \text{ s.t. } \sigma \in \mathcal{L}(A_\varphi)} \Psi(\sigma) \bowtie p$$

where A_φ is the (minimal) DFA capturing the language of φ . $\triangleleft$

Example 16. Consider the order-to-cash process of our running example. We introduce a ProbDeclare specification D_{order} containing three probabilistic constraints:

- Φ_{pr} is a crisp constraint expressing that the *not coexistence* between pay and ack reject (namely that pay and ack reject cannot both occur in the same trace) must hold with probability 1;
- Φ_{op} is a probabilistic constraint expressing that the *response* from open to pay (namely that whenever open occurs then pay later occurs as well) must hold with a probability $\geq \frac{1}{20}$;
- Φ_{or} is a probabilistic constraint expressing that the *response* from open to ack reject (namely that whenever open occurs then ack reject later occurs as well) must hold with a probability $\leq \frac{1}{4}$. $\triangleleft$

7.2. Checking stochastic compliance

To check whether a livelock-free bounded LSP stochastically complies with a ProbDeclare specification, we simply verify whether the stochastic language induced by the LSP indeed satisfies all probabilistic constraints with their probability conditions.

Problem 5 ($\text{S-COMPLIES}(\mathcal{N}, D)$).

Input: livelock-free bounded LSP $\mathcal{N}$, ProbDeclare specification D ;

Output: yes if the induced stochastic language of $\mathcal{N}$ (as per Definition 25) satisfies D in the sense of Definition 33, no otherwise. $\triangleleft$

We can solve this problem through iterated calls to the SPEC-PROB problem, using the DFA of each probabilistic constraint, and checking whether the returned probability satisfies the probability condition associated to that constraint.

Theorem 6. Consider a livelock-free bounded LSP $\mathcal{N}$ and a ProbDeclare specification D . Then $\text{S-COMPLIES}(\mathcal{N}, D)$ returns yes if and only if for every probabilistic constraint $\langle \varphi, \bowtie, p \rangle \in D$, we have that $\text{SPEC-PROB}(\mathcal{N}, A_\varphi) \bowtie p$. $\triangleleft$

Proof. Fix a probabilistic constraint $\langle \varphi, \bowtie, p \rangle \in D$. By Definitions 33 and 25, we have that

$$\sum_{\sigma \in A^* \text{ s.t. } \sigma \in \mathcal{L}(A_\varphi)} \mathbb{P}_{\mathcal{N}}(\sigma) = \sum_{\sigma \text{ model trace of } \mathcal{N} \text{ s.t. } \sigma \in \mathcal{L}(A_\varphi)} \mathbb{P}_{\mathcal{N}}(\sigma)$$

Indeed, notice that the stochastic language induced by $\mathcal{N}$ is so that $\mathbb{P}_{\mathcal{N}}(\sigma) > 0$ only if σ is a model trace of $\mathcal{N}$. This sum is, in turn, equal to $\mathbb{P}_{\mathcal{N}}(A_\varphi)$, which, by definition, is the output of $\text{SPEC-PROB}(\mathcal{N}, A_\varphi)$. $\triangleleft$

Example 17. Consider $\mathcal{N}_{\text{order}}$ under the assumption that all weights are equal (i.e., the working hypothesis of Example 10). We now show that under this assumption $\mathcal{N}_{\text{order}}$ stochastically complies with the ProbDeclare specification D_{order} from Example 16. We can do so by simply applying Theorem 6. To witness semantic compliance, we notice the following:

- Every model trace of $\mathcal{N}_{\text{order}}$ contains either pay, or ack reject, or none of the two, and thus satisfies the *not coexistence* between pay and ack reject; by 3, we know that the sum of the probability values of all model traces of $\mathcal{N}_{\text{order}}$ is indeed 1 (as requested by the probabilistic constraint $\Phi_{pr} \in D_{\text{order}}$).
- Considering the structure of $RG(\mathcal{N}_{\text{order}})$, we have that model traces satisfying the *response* from open to pay are simply those that culminate in final marking $[q_h]$. By Example 10, we know that the overall probability of such model traces is $\frac{1}{11}$, which is indeed $\geq \frac{1}{20}$ (as requested by the probabilistic constraint $\Phi_{op} \in D_{\text{order}}$);
- Considering the structure of $RG(\mathcal{N}_{\text{order}})$, we have that model traces satisfying the *response* from open to reject are simply

those that culminate in final marking $[q_r]$. By Example 10, we know that the overall probability of such model traces is $\frac{6}{11}$, which is indeed $\leq \frac{1}{4}$ (as requested by the probabilistic constraint $\Phi_{or} \in D_{\text{order}}$). $\triangleleft$

In case S-COMPLIES returns a negative answer, the standard Earth Mover's Distance can be used to measure the deviation between the probability distribution by the livelock-free bounded LSP under scrutiny, and what expected by the probabilistic constraint in the reference ProbDeclare specification. To do so, one has to follow the approach in [18], computing the different *scenarios* induced by the ProbDeclare specification and their probabilities. This realises a form of *stochastic delta analysis*.

8. A practical application & its evaluation

To illustrate the practical applicability and feasibility of the verification problem, we implemented it, and used this implementation in a previously incomplete stochastic conformance measure.

8.1. Implementation

Our solutions to the trace and temporal properties probability have been implemented as plug-ins of the ProM framework [28], as part of the StochasticLabelledPetriNets package. This package provides stochastic PNPs as first-class objects in ProM, an import plug-in, an export plug-in and a visualiser. The package further contains an implementation of the trace probability approach (cf. Section 6.3): given a trace, this approach constructs an explicit product stochastic transition system, after which a linear problem is solved using Lpsolve [43] to obtain the trace probability. Besides a trace, the implementation provides the machinery to accept any type of DFA.

This code is intended for developers, however we used it to complete an existing method: Unit Earth Movers' Stochastic Conformance (uEMSC) [22].¹ This stochastic conformance checking measure expresses the difference between a stochastic process model and an event log, which are for our purposes seen as two distributions of traces. Intuitively, the measure indicates the amount of earth (i.e. probability mass) that needs to be moved to transform one of the distributions into the other. From [22], uEMSC of a log L and a stochastic process model $\mathcal{N}$ is $1 - \sum_{\sigma \in L} \max(L(\sigma) - \mathbb{P}_{\mathcal{N}}(\sigma), 0)$, in which $L(\sigma)$ denotes the likelihood of trace σ in L . Before, approximations of $\mathbb{P}_{\mathcal{N}}(\sigma)$ were necessary to compute this otherwise rather simple function [15,17].

The – now complete – uEMSC implementation is available in the ProM framework [28] as the “Compute unit Earth-movers' stochastic conformance (log-model)” plug-in, part of the “EarthMoversStochastic-ConformanceChecking” package.

8.2. Applicability and feasibility

In this section, we empirically show the applicability of our solution of the verification problem. We do this by using our solution as a trace-probability predictor method in the uEMSC method, which was previously incomplete. Unit Earth Movers' Stochastic Conformance (uEMSC) takes an event log and a stochastic process model (in our cases, a labelled stochastic Petri net). uEMSC considers both the log and the stochastic model as a distribution over traces. Then, it considers the earth movers' distance of these distributions: how much effort does it take to transform one distribution into the other, in terms of moved probability mass and the distance over which it has to be moved. The uEMSC measure considers a unit distance between traces – traces are either equivalent or different – and thus is a measure of the agreement in stochastic languages of log and model. In uEMSC, our solution of the

¹ Notice that uEMSC is then complete for bounded models only.

Table 1
Details of the set-up of the feasibility evaluation.

(a) Logs.		(b) Discovery techniques for control flow.	
BPIC11 [44]	BPIC11	Inductive miner - infrequent (0.8) [45] transformed to a Petri net	IMf
BPIC12-approvals [46]	BPIC12-a	Directly follows model miner (0.8) [47] transformed to a Petri net	DFM
BPIC12-approvals and work [46]	BPIC12-aw	Flower model – a model consisting of any behaviour of the observed alphabet	FM
BPIC13-closed problems [48]		(c) Stochastic discovery techniques.	
BPIC13-incidents [49]	BPIC13-cp	Frequency-based estimator [9]	FBE
BPIC13-open problems [50]	BPIC13-i	Bill Clinton estimator [9]	BCE
BPIC17-offers [51]	BPIC17-op	Alignment-based estimator [9]	ABE
BPIC18-parcel document [52]	BPIC18-o	(d) Measures.	
BPIC20-domestic declarations [53]	BPIC18-6	Number of traces, events, activities in log	log
BPIC20-international declarations [53]	BPIC20-dd	Number of transitions, silent transitions in control-flow model	transitions
BPIC20-prepaid travel costs [53]	BPIC20-id	Earth Movers' conformance checking [15] model sample up to 50 000 traces	EMSC
BPIC20-request for payment [53]	BPIC20-pt	Unit Earth Movers' conformance checking (Section 8.1)	uEMSC
MIMIC services [54] [55, A]	BPIC20-rf		
MIMIC transfers [54] [55, A]	mimic-serv		
Road fines [56]	mimic-trans		
Sepsis cases [57]	roadfines		
	sepsis		

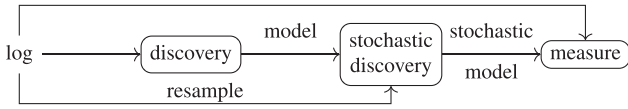


Fig. 10. Feasibility evaluation set-up.

verification problem was included to assess the probability of a given trace in the model.

8.2.1. Set-up

In this section, we illustrate the practical applicability of this implementation, using an experiment, summarised in Fig. 10 and detailed in Table 1. We take a number of publicly available real-life event logs, and apply a control flow discovery technique to obtain a process model (guaranteed bounded and without stochastic perspective). To avoid measuring on training data, the event log is resampled 10 times. To each resampled log and the discovered control-flow model, we apply a stochastic process discovery technique to obtain a stochastic process model. Then, the quality of this model is assessed with respect to the original log using several measures.

8.2.2. Results

Fig. 11 shows a summary of the results; Table A.2 in Appendix shows the results in detail.

8.2.3. Discussion

Measured values. When comparing uEMSC with EMSC on the values in Fig. 11(a), the first observation is that uEMSC has many more low measures, even quite a few being around 0, and none below the dashed equivalence line. This is due to the trace-based nature of uEMSC vs. the event-based nature of EMSC: for a too-low EMSC value, uEMSC stays close to 0, as only events can be mapped but the probabilistic overlap in traces is negligible.

An exception, visible in Fig. 11(a), are some points where EMSC is high (over 0.8) but uEMSC is still 0; these points are from BPIC2018-6 DFM FBE, BPIC2018-6 DFM BCE, BPIC20-id DFM FBE and BPIC20-id DFM BCE. All of these involve the FBE and BCE stochastic discovery techniques, while ABE does not exhibit this pattern. A reason can be found in that FBE puts a weight of 1 for each silent transition and BCE puts a value close to 1 on the silent transitions.

In DFM models, silent transitions are only used for termination, such that these transitions with a weight close to 1 are competing against visible transitions with a weight close to their execution frequency

(which is in the 10-thousands). Thus, on the one hand uEMSC assigns a low score to any trace in these models, which gets rounded to 0 eventually. EMSC on the other hand is much less sensitive to this single weight as it takes a sample of behaviour of the model.

Run time. Both uEMSC and EMSC were feasible for the vast majority of log-model combinations of this experiment: as can be seen in Fig. 11(b), for most logs uEMSC's run time was at most a few seconds, while EMSC required up to a day.

There are exceptions: BPIC11, for which uEMSC required up to half an hour or did not return a result in our main memory available (40GB). In particular for BPIC11 IMf, there were 345 silent transitions, leading to a too-large state space to be explored in the product stochastic transition system. EMSC takes in general much longer, with BPIC12-AW IMf FBE and BCE running out of memory.

A notable case where uEMSC ran longer than EMSC is mimic-serv IMf FBE and mimic-serv IMf BCE, where uEMSC takes 5–6 s, while EMSC only needs a few milliseconds. Fig. 12 shows this model. In this model, there is firstly a choice between on the one hand executing the visible transition NB with weight 7819 and on the other hand a large block of parallel behaviour, which starts with a silent transition of weight 1. EMSC samples only one trace from this model, with probability mass $\frac{7819}{7820}$. This leads to an answer quickly, whereas uEMSC performs an analysis on the large product stochastic transition system due to the 55 partly concurrent silent transitions.

9. Conclusion

We have introduced (bounded) labelled stochastic processes, an extension of generalised stochastic Petri nets with (possibly duplicated) labels and silent transitions, and defined their finite-trace execution semantics. We have shown how different analysis techniques combining behaviours and their probabilities can be solved through a combination of techniques from Markov chains and their qualitative verification, suitably adapted and refined to deal with this challenging setting. All techniques are implemented in the StochasticLabelledPetriNets plug-in of ProM.

Our approach lazily handles silent transitions when intersecting the set of paths characterised by a labelled stochastic process (including silent steps), with the traces of interest (predicating on visible activities only). To specify traces of interest, we employ deterministic finite-state automata. By recalling that temporal logics on finite traces such as LTL_f and LDL_f have an automata-theoretic characterisation based on deterministic finite-state automata [36,37], our approach provides, for

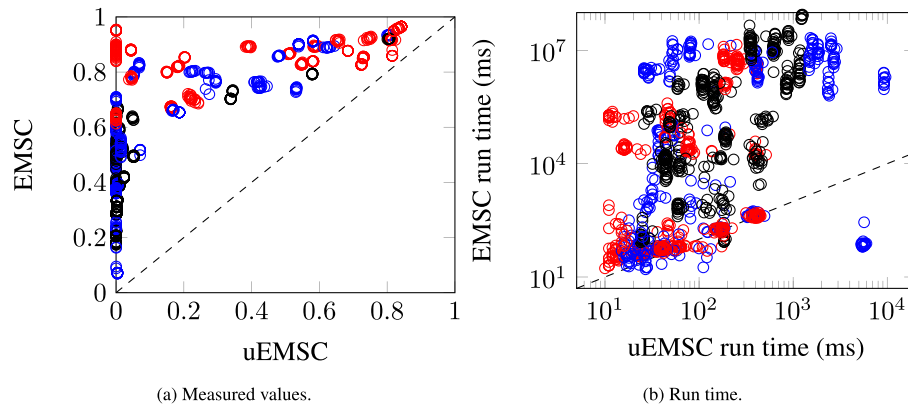


Fig. 11. Results of our feasibility experiment (only complete logs included). Dashed lines denote equality. Colours indicate IMf, DFM and FM.

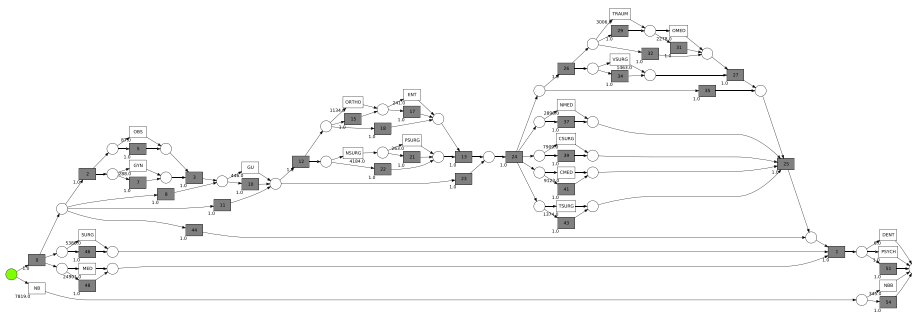


Fig. 12. mimic-serv IMf BFE model.

the first time, an technique for model checking stochastic processes against such finite-trace LTL_f/LDL_f specifications.

When computing the probability of model traces, this is in contrast with ad-hoc probability computation techniques developed in the context of stochastic automata with ϵ -moves in natural language processing [26]. There, silent steps are filtered out in an eager way, using ad-hoc algorithms. A natural follow-up of this work is then to comparatively assess our lazy approach with such eager algorithm. On a similar note, we intend to conduct a more extensive experimental evaluation, with the aim of assessing the usage of well-established state-of-the-art probabilistic model checkers (such as Prism² and Storm³) to deal with our analysis tasks.

Notably, our approach seamlessly generalise to the case where weights/rates are not exact but come with a confidence interval: one just needs to transform the system of equations used for computing outcome probabilities into a corresponding system of inequalities. The next step is then to consider this and other extensions of the model, in particular looking into richer nets with nondeterministic transition effects and their mapping to Markov decision processes, nets with non-Markovian behaviours, and nets equipped with different distributions on durations/delays. This calls for a fine-grained analysis of the same problems studied here in such richer settings, where often analytical methods are out of reach and have to be replaced by approximate/numerical techniques or simulation.

CRediT authorship contribution statement

Sander J.J. Leemans: Validation, Software, Investigation, Conceptualization. **Fabrizio Maria Maggi:** Software, Investigation, Conceptualization. **Marco Montali:** Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data from public repositories.

Acknowledgements

This work has been partially funded by the UNIBZ project ADAPTERS, and the PRIN MIUR project PINPOINT Prot. 2020FNEB27. We thank Werner Nutt for interesting discussions on exponential random variables and their interplay.

Appendix. Evaluation results

See Table A.2.

² <https://www.prismmodelchecker.org>.

³ <https://www.stormchecker.org>.

Table A.2
Results of the evaluation.

Log	Discovery	sto.dis.	Log			Transitions		EMSC		uEMSC	
			Traces	Events	Activities	Silent		Time (ms)		Time (ms)	
BPIC11	IMf	FBE	1 143	150 291	624	580	345	–	–	–	–
		BCE	1 143	150 291	624	580	345	–	–	–	–
		ABE	–	–	–	–	–	–	–	–	–
	DFM	FBE	1 143	150 291	624	4295	35	0.22	307 002	0.0000	69 541
		BCE	1 143	150 291	624	4295	35	0.22	292 145	0.0000	69 110
		ABE	–	–	–	–	–	–	–	–	–
	FM	FBE	1 143	150 291	624	627	3	0.19	11 724 000	0.0032	1 809 636
		BCE	1 143	150 291	624	627	3	0.19	12 497 634	0.0032	1 762 811
		ABE	1 143	150 291	624	627	3	0.14	104 537	0.0003	1 801 286
BPIC12-a	IMf	FBE	13 087	60 849	10	15	5	0.65	53	0.1870	42
		BCE	13 087	60 849	10	15	5	0.65	63	0.1870	42
		ABE	13 087	60 849	10	15	5	0.86	62	0.4801	51
	DFM	FBE	13 087	60 849	10	16	3	0.85	53	0.7295	42
		BCE	13 087	60 849	10	16	3	0.85	57	0.7295	41
		ABE	13 087	60 849	10	16	3	0.96	56	0.8327	55
	FM	FBE	13 087	60 849	10	13	3	0.33	8 135	0.0016	59
		BCE	13 087	60 849	10	13	3	0.33	6 131	0.0016	46
		ABE	13 087	60 849	10	13	3	0.53	13 871	0.0017	42
BPIC12-aw	IMf	FBE	13 087	204 638	16	44	28	–	–	0.0000	1 154
		BCE	13 087	204 638	16	44	28	–	–	0.0000	1 084
		ABE	13 087	204 638	16	44	28	–	–	0.0025	1 794
	DFM	FBE	13 087	204 638	16	48	6	0.84	24 529	0.0000	723
		BCE	13 087	204 638	16	48	6	0.84	24 714	0.0000	685
		ABE	13 087	204 638	16	48	6	0.79	3 249 424	0.1395	732
	FM	FBE	13 087	204 638	16	19	3	–	–	0.0000	2 547
		BCE	13 087	204 638	16	19	3	–	–	0.0000	2 612
		ABE	13 087	204 638	16	19	3	–	–	0.0000	2 591
BPIC13-cp	IMf	FBE	1 487	6 660	7	15	9	0.51	25	0.0143	28
		BCE	1 487	6 660	7	15	9	0.55	42	0.0142	17
		ABE	1 487	6 660	7	15	9	0.76	130	0.4175	21
	DFM	FBE	1 487	6 660	7	13	1	0.95	26 983	0.0002	15
		BCE	1 487	6 660	7	13	1	0.95	24 674	0.0002	18
		ABE	1 487	6 660	7	13	1	0.91	137	0.6547	15
	FM	FBE	1 487	6 660	7	10	3	0.49	708	0.0517	59
		BCE	1 487	6 660	7	10	3	0.49	671	0.0517	66
		ABE	1 487	6 660	7	10	3	0.63	1 223 077	0.0491	76
BPIC13-i	IMf	FBE	7 554	65 533	13	31	20	0.39	5 605 563	–0.0000	1 523
		BCE	7 554	65 533	13	31	20	0.46	16 199 640	0.0000	1 621
		ABE	7 554	65 533	13	31	20	0.79	8 709 087	0.0527	1 871
	DFM	FBE	7 554	65 533	13	24	2	0.88	3 931 804	0.0442	253
		BCE	7 554	65 533	13	24	2	0.88	4 931 212	0.0442	264
		ABE	7 554	65 533	13	24	2	0.80	6 735 050	0.1500	257
	FM	FBE	7 554	65 533	13	16	3	0.34	1 139 508	0.0048	843
		BCE	7 554	65 533	13	16	3	0.34	1 782 487	0.0048	875
		ABE	7 554	65 533	13	16	3	0.67	25 740 542	0.0050	895
BPIC13-op	IMf	FBE	819	2 351	5	16	11	0.39	24	0.0056	37
		BCE	819	2 351	5	16	11	0.52	53	0.0709	30
		ABE	819	2 351	5	16	11	0.75	845	0.4269	29
	DFM	FBE	819	2 351	5	17	5	0.70	185 299	0.2242	11
		BCE	819	2 351	5	17	5	0.71	169 801	0.2242	13
		ABE	819	2 351	5	17	5	0.92	173	0.7456	10
	FM	FBE	819	2 351	5	8	3	0.70	127	0.3383	24
		BCE	819	2 351	5	8	3	0.70	145	0.3383	26
		ABE	819	2 351	5	8	3	0.73	209 141	0.3440	33
BPIC17-o	IMf	FBE	42 995	193 849	8	9	1	0.90	186	0.5390	170
		BCE	42 995	193 849	8	9	1	0.90	185	0.5390	162
		ABE	42 995	193 849	8	9	1	0.91	192	0.5811	162
	DFM	FBE	42 995	193 849	8	10	3	0.96	183	0.8419	168
		BCE	42 995	193 849	8	10	3	0.96	181	0.8419	171
		ABE	42 995	193 849	8	10	3	0.96	186	0.8419	168
	FM	FBE	42 995	193 849	8	11	3	0.31	971	0.0023	162
		BCE	42 995	193 849	8	11	3	0.31	1 693	0.0023	150
		ABE	42 995	193 849	8	11	3	0.50	14 026	0.0026	172

(continued on next page)

Table A.2 (continued).

Log	Discovery	sto.dis.	Log			Transitions		EMSC		uEMSC	
			Traces	Events	Activities	Silent		Time (ms)		Time (ms)	
BPIC18-6	IMf	FBE	14 750	132 963	10	24	15	0.55	1 225 049	0.0003	2 394
		BCE	14 750	132 963	10	24	15	0.59	14 419 026	0.0003	2 556
		ABE	14 750	132 963	10	24	15	0.83	5 879 827	0.0684	2 729
	DFM	FBE	14 750	132 963	10	16	1	0.88	20 288	0.0000	410
		BCE	14 750	132 963	10	16	1	0.89	18 087	0.0000	402
		ABE	14 750	132 963	10	16	1	0.85	3 547 940	0.2025	21 579
	FM	FBE	14 750	132 963	10	13	3	0.28	4 074 955	0.0008	1 186
		BCE	14 750	132 963	10	13	3	0.28	3 589 177	0.0008	1 181
		ABE	14 750	132 963	10	13	3	0.64	85 640 171	0.0009	1 237
BPIC20-dd	IMf	FBE	10 500	56 437	17	33	17	0.40	25 320	0.0001	41
		BCE	10 500	56 437	17	33	17	0.49	72 573	0.0001	46
		ABE	10 500	56 437	17	33	17	0.93	53	0.8049	41
	DFM	FBE	10 500	56 437	17	9	1	0.84	50	0.5807	35
		BCE	10 500	56 437	17	9	1	0.84	50	0.5807	37
		ABE	10 500	56 437	17	9	1	0.92	61	0.8079	36
	FM	FBE	10 500	56 437	17	20	3	0.29	16 330	0.0019	60
		BCE	10 500	56 437	17	20	3	0.29	15 637	0.0019	64
		ABE	10 500	56 437	17	20	3	0.55	169 561	0.0012	65
BPIC20-id	IMf	FBE	6 449	72 151	34	66	36	0.51	14 278 269	−0.0000	99
		BCE	6 449	72 151	34	66	36	0.53	8 095 583	−0.0000	80
		ABE	6 449	72 151	34	66	36	0.78	1 494	0.2690	108
	DFM	FBE	6 449	72 151	34	51	3	0.86	22 502	0.0000	86
		BCE	6 449	72 151	34	51	3	0.87	21 782	0.0000	75
		ABE	6 449	72 151	34	51	3	0.82	41 291	0.1830	96
	FM	FBE	6 449	72 151	34	37	3	0.18	10 800 412	0.0000	622
		BCE	6 449	72 151	34	37	3	0.18	11 029 039	0.0000	618
		ABE	6 449	72 151	34	37	3	0.42	19 241 284	0.0000	665
BPIC20-pt	IMf	FBE	2 099	18 246	29	63	37	0.46	2 710 449	0.0000	30
		BCE	2 099	18 246	29	63	37	0.46	2 975 697	0.0000	27
		ABE	2 099	18 246	29	63	37	0.80	2 770	0.2248	44
	DFM	FBE	2 099	18 246	29	31	2	0.67	61	0.1624	13
		BCE	2 099	18 246	29	31	2	0.67	63	0.1624	12
		ABE	2 099	18 246	29	31	2	0.89	62	0.3910	13
	FM	FBE	2 099	18 246	29	32	3	0.22	1 151 046	0.0006	109
		BCE	2 099	18 246	29	32	3	0.22	1 250 116	0.0006	118
		ABE	2 099	18 246	29	32	3	0.51	1 637 301	0.0002	127
BPIC20-rf	IMf	FBE	6 886	36 796	19	29	13	0.37	55	−0.0000	26
		BCE	6 886	36 796	19	29	13	0.37	80	−0.0000	22
		ABE	6 886	36 796	19	29	13	0.89	39	0.6262	32
	DFM	FBE	6 886	36 796	19	12	1	0.83	37	0.5483	24
		BCE	6 886	36 796	19	12	1	0.83	51	0.5483	21
		ABE	6 886	36 796	19	12	1	0.92	44	0.8118	23
	FM	FBE	6 886	36 796	19	22	3	0.30	16 568	0.0024	44
		BCE	6 886	36 796	19	22	3	0.30	15 904	0.0024	47
		ABE	6 886	36 796	19	22	3	0.55	116 739	0.0018	51
mimic-serv	IMf	FBE	58 926	73 343	20	55	35	0.07	71	0.0038	5 433
		BCE	58 926	73 343	20	55	35	0.07	94	0.0038	5 784
		ABE	58 926	73 343	20	55	35	0.74	1 697 889	0.5318	9 265
	DFM	FBE	58 926	73 343	20	19	9	0.83	65	0.7241	63
		BCE	58 926	73 343	20	19	9	0.83	64	0.7241	57
		ABE	58 926	73 343	20	19	9	0.86	66	0.8150	79
	FM	FBE	58 926	73 343	20	23	3	0.79	49 588	0.5790	204
		BCE	58 926	73 343	20	23	3	0.79	50 026	0.5790	193
		ABE	58 926	73 343	20	23	3	0.92	114	0.8051	185
mimic-trans	IMf	FBE	3 019	13 411	18	37	19	0.21	10 231 861	0.0003	53
		BCE	3 019	13 411	18	37	19	0.22	3 675 148	0.0003	60
		ABE	3 019	13 411	18	37	19	0.66	74 478	0.1681	66
	DFM	FBE	3 019	13 411	18	37	4	0.87	219 124	0.5112	49
		BCE	3 019	13 411	18	37	4	0.87	106 450	0.5112	46
		ABE	3 019	13 411	18	37	4	0.89	184	0.5877	52
	FM	FBE	3 019	13 411	18	21	3	0.39	620 518	0.0100	145
		BCE	3 019	13 411	18	21	3	0.39	593 890	0.0100	145
		ABE	3 019	13 411	18	21	3	0.59	1 309 639	0.0098	145
Roadfines	IMf	FBE	150 370	561 470	11	24	13	0.54	449	0.0139	394
		BCE	150 370	561 470	11	24	13	0.54	443	0.0104	361
		ABE	150 370	561 470	11	24	13	0.77	422	0.2940	392

(continued on next page)

Table A.2 (continued).

Log	Discovery	sto.dis.	Log			Transitions		EMSC		uEMSC	
			Traces	Events	Activities	Silent		Time (ms)		Time (ms)	
Sepsis	DFM	FBE	150 370	561 470	11	9	3	0.88	427	0.6840	408
		BCE	150 370	561 470	11	9	3	0.88	407	0.6840	388
		ABE	150 370	561 470	11	9	3	0.95	419	0.8196	417
	FM	FBE	150 370	561 470	11	14	3	0.42	13 637	0.0236	408
		BCE	150 370	561 470	11	14	3	0.42	27 969	0.0236	426
		ABE	150 370	561 470	11	14	3	0.52	146 820	0.0193	502
	IMf	FBE	1 050	15 214	16	29	15	0.51	8 246 699	−0.0000	406
		BCE	1 050	15 214	16	29	15	0.60	4 252 316	0.0000	408
		ABE	1 050	15 214	16	29	15	0.70	1 926 387	0.0003	420
	DFM	FBE	1 050	15 214	16	77	10	0.63	6 517 108	0.0000	188
		BCE	1 050	15 214	16	77	10	0.64	7 041 945	0.0000	188
		ABE	1 050	15 214	16	77	10	0.78	1 253 417	0.0451	191
	FM	FBE	1 050	15 214	16	19	3	0.21	18 589 789	0.0000	351
		BCE	1 050	15 214	16	19	3	0.21	20 377 324	0.0000	366
		ABE	1 050	15 214	16	19	3	0.50	8 652 211	0.0000	364

References

- [1] S.O. Natkin, *Les Réseaux de Petri Stochastiques et leur Application de l'Evaluation des Systemes Informatiques*, 1980.
- [2] M.K. Molloy, On the integration of delay and throughput measures in distributed processing models, 1982.
- [3] M.K. Molloy, Performance analysis using stochastic Petri nets, *IEEE Trans. Comput.* 31 (1982) 913–917.
- [4] M.A. Marsan, G. Conte, G. Balbo, A class of generalized stochastic Petri nets for the performance evaluation of multiprocessor systems, *ACM TOCS* 2 (2) (1984) 93–122.
- [5] M.A. Marsan, Stochastic Petri nets: an elementary introduction, in: *European Workshop on Applications and Theory in Petri Nets*, Springer, 1988, pp. 1–29.
- [6] M. Ajmone Marsan, G. Balbo, G. Conte, S. Donatelli, G. Franceschinis, *Modelling with Generalized Stochastic Petri Nets*, in: *Wiley Series in Parallel Computing*, John Wiley and Sons, 1995.
- [7] A. Burke, S. Leemans, M. Wynn, Stochastic process discovery by weight estimation, in: *PQMI*, 2020.
- [8] A. Rogge-Solti, W.M.P. van der Aalst, M. Weske, Discovering stochastic Petri nets with arbitrary delay distributions from event logs, in: *BPMW13*, 2013, pp. 15–27.
- [9] A. Burke, S.J.J. Leemans, M.T. Wynn, Stochastic process discovery by weight estimation, in: S.J.J. Leemans, H. Leopold (Eds.), *Process Mining Workshops - ICPM 2020 International Workshops*, Padua, Italy, October 5–8, 2020, Revised Selected Papers, in: *Lecture Notes in Business Information Processing*, vol. 406, Springer, 2020, pp. 260–272, http://dx.doi.org/10.1007/978-3-030-72693-5_20.
- [10] A. Burke, S.J.J. Leemans, M.T. Wynn, Discovering stochastic process models by reduction and abstraction, in: D. Buchs, J. Carmona (Eds.), *Application and Theory of Petri Nets and Concurrency - 42nd International Conference, PETRI NETS 2021, Virtual Event, June 23–25, 2021, Proceedings*, in: *Lecture Notes in Computer Science*, vol. 12734, Springer, 2021, pp. 312–336, http://dx.doi.org/10.1007/978-3-030-76983-3_16.
- [11] A. Rogge-Solti, R. Mans, W. van der Aalst, M. Weske, Repairing event logs using timed process models, in: *OTM 2013 Workshops*, in: *LNCS*, vol. 8186, Springer, 2013, pp. 705–708.
- [12] A. Polyvyanyy, A.A. Kalenkova, Monotone conformance checking for partially matching designed and observed processes, in: *ICPM*, 2019, pp. 81–88.
- [13] A. Polyvyanyy, A. Solti, M. Weidlich, C. Di Ciccio, J. Mendling, Monotone precision and recall measures for comparing executions and specifications of dynamic systems, *ACM Trans. Softw. Eng. Methodol.* 29 (3) (2020) 17:1–17:41.
- [14] H. Alkhamash, A. Polyvyanyy, A. Moffat, L. García-Bañuelos, Entropic relevance: A mechanism for measuring stochastic process models discovered from event data, *Inf. Syst.* 107 (2022) 101922, <http://dx.doi.org/10.1016/j.is.2021.101922>.
- [15] S.J.J. Leemans, W.M.P. van der Aalst, T. Brockhoff, A. Polyvyanyy, Stochastic process mining: Earth movers' stochastic conformance, *Inf. Syst.* 102 (2021) 101724.
- [16] S.J.J. Leemans, A. Polyvyanyy, Stochastic-aware conformance checking: An entropy-based approach, in: *Advanced Information Systems Engineering*, Springer, 2020, pp. 217–233.
- [17] G. Bergami, F.M. Maggi, M. Montali, R. Peñañoza, Probabilistic trace alignment, in: *ICPM, IEEE*, 2021, pp. 9–16.
- [18] A. Alman, F.M. Maggi, M. Montali, R. Peñañoza, Probabilistic declarative process mining, *Inf. Syst.* 109 (2022) 102033.
- [19] S.J.J. Leemans, D. Fahland, W.M.P. van der Aalst, Discovering block-structured process models from event logs - A constructive approach, in: J.M. Colom, J. Desel (Eds.), *Application and Theory of Petri Nets and Concurrency - 34th International Conference, PETRI NETS 2013, Milan, Italy, June 24–28, 2013. Proceedings*, in: *Lecture Notes in Computer Science*, vol. 7927, Springer, 2013, pp. 311–329, http://dx.doi.org/10.1007/978-3-642-38697-8_17.
- [20] J.C.A.M. Buijs, B.F. van Dongen, W.M.P. van der Aalst, A genetic algorithm for discovering process trees, in: *Proceedings of the IEEE Congress on Evolutionary Computation, CEC 2012, Brisbane, Australia, June 10–15, 2012, IEEE*, 2012, pp. 1–8, <http://dx.doi.org/10.1109/CEC.2012.6256458>.
- [21] A. Augusto, R. Conforti, M. Dumas, M. La Rosa, A. Polyvyanyy, Split miner: automated discovery of accurate and simple business process models from event logs, *Knowl. Inf. Syst.* 59 (2) (2019) 251–284.
- [22] S.J.J. Leemans, A.F. Syring, W.M.P. van der Aalst, Earth movers' stochastic conformance checking, in: *Proceedings of the BPM Forum*, Vol. 360, 2019, pp. 127–143.
- [23] G. Chiola, M.A. Marsan, G. Balbo, G. Conte, Generalized stochastic Petri nets: A definition at the net level and its implications, *IEEE Trans. Softw. Eng.* 19 (2) (1993) 89–107.
- [24] C.M. Grinstead, J.L. Snell, *Introduction to Probability*, second ed., American Mathematical Society, 1997.
- [25] C. Baier, J. Katoen, *Principles of Model Checking*, MIT Press, 2008.
- [26] T. Hanneforth, C. De La Higuera, Epsilon-removal by loop reduction for finite-state automata over complete semirings, *Stud. Gramm.* 72 (2010) 297–312.
- [27] S.J.J. Leemans, F.M. Maggi, M. Montali, Reasoning on labelled Petri nets and their dynamics in a stochastic setting, in: C. Di Ciccio, R.M. Dijkman, A. del-Río-Ortega, S. Rinderle-Ma (Eds.), *Proceedings of the 20th International Conference on Business Process Management, BPM 2022*, in: *Lecture Notes in Computer Science*, vol. 13420, Springer, 2022, pp. 324–342, http://dx.doi.org/10.1007/978-3-031-16103-2_22.
- [28] B.F. van Dongen, et al., The ProM framework: A new era in process mining tool support, in: *Applications and Theory of Petri Nets*, Vol. 3536, 2005, pp. 444–454.
- [29] G. Bergami, F.M. Maggi, M. Montali, R. Peñañoza, A tool for probabilistic trace alignments, in: *CAiSE Forum*, Springer, 2021.
- [30] A. Burke, S.J.J. Leemans, M.T. Wynn, W.M.P. van der Aalst, A.H.M. ter Hofstede, Stochastic process model-log quality dimensions: An experimental study, in: A. Burattin, A. Polyvyanyy, B. Weber (Eds.), *Proceedings of the 4th International Conference on Process Mining, ICPM 2022, IEEE*, 2022, pp. 80–87.
- [31] R. Bruni, H.C. Melgratti, U. Montanari, Concurrency and probability: Removing confusion, compositionally, *Log. Methods Comput. Sci.* 15 (4) (2019).
- [32] M. Ajmone Marsan, Stochastic Petri nets: an elementary introduction, in: G. Rozenberg (Ed.), *Advances in Petri Nets 1989*, in: *LNCS*, vol. 424, Springer, 1988, pp. 1–29.
- [33] J. Desel, W. Reisig, Place/transition Petri nets, in: *Lectures on Petri Nets I: Basic Models: Advances in Petri Nets*, Springer, 1998, pp. 122–173.
- [34] R. Durrett, *Essentials of Stochastic Processes*, third ed., Springer, 2016.
- [35] J. Hopcroft, R. Motwani, J. Ullman, *Introduction to Automata Theory, Languages, and Computation*, third ed., 2007.
- [36] G. De Giacomo, M.Y. Vardi, Linear temporal logic and linear dynamic logic on finite traces, in: *Proceedings IJCAI, AAAI Press*, 2013.
- [37] G. De Giacomo, R. De Masellis, F.M. Maggi, M. Montali, Monitoring constraints and metaconstraints with temporal logics on finite traces, *ACM TOSEM* (2022).
- [38] M. Pesic, H. Schonenberg, W.M.P. van der Aalst, DECLARE: Full support for loosely-structured processes, in: *Proceedings of the 11th IEEE International Enterprise Distributed Object Computing Conf., EDOC, IEEE Computer Society*, 2007, pp. 287–300.

- [39] M. Montali, M. Pesic, W.M.P. van der Aalst, F. Chesani, P. Mello, S. Storari, Declarative specification and verification of service choreographies, *ACM Trans. Web* 4 (1) (2010).
- [40] F.M. Maggi, M. Montali, R. Peñaloza, A. Alman, Extending temporal business constraints with uncertainty, in: *Proceedings of BPM*, in: *LNCs*, vol. 12168, Springer, 2020, pp. 35–54.
- [41] C. Di Ciccio, M. Montali, Declarative process specifications: Reasoning, discovery, monitoring, in: W.M.P. van der Aalst, J. Carmona (Eds.), *Process Mining Handbook*, in: *Lecture Notes in Business Information Processing*, vol. 448, Springer, 2022, pp. 108–152.
- [42] F.M. Maggi, M. Montali, R. Peñaloza, Temporal logics over finite traces with uncertainty, in: *Proceedings of the 34th AAAI Conference on Artificial Intelligence*, AAAI 2020, 2020, pp. 10218–10225.
- [43] M. Berkelaar, K. Eikland, P. Notebaert, *lp_solve 5.5*, open source (mixed-integer) linear programming system, 2004, Software. <http://lpsolve.sourceforge.net/5.5/>, available at <<http://lpsolve.sourceforge.net/5.5/>>. Last accessed Dec, 18 2009.
- [44] B. van Dongen, Real-life event logs - Hospital log, 2011, <http://dx.doi.org/10.4121/uuid:d9769f3d-0ab0-4fb8-803b-0d1120ffcf54>, https://data.4tu.nl/articles/dataset/Real-life_event_logs_-_Hospital_log/12716513.
- [45] S.J.J. Leemans, D. Fahland, W.M.P. van der Aalst, Discovering block-structured process models from event logs containing infrequent behaviour, in: *BPM Workshops*, in: *LNBIP*, vol. 171, 2013.
- [46] B. van Dongen, BPI challenge 2012, 2012, <http://dx.doi.org/10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f>, https://data.4tu.nl/articles/dataset/BPI_Challenge_2012/12689204.
- [47] S.J.J. Leemans, E. Poppe, M.T. Wynn, Directly follows-based process mining: Exploration & a case study, in: *ICPM*, 2019.
- [48] W. Steeman, BPI challenge 2013, closed problems, 2013, <http://dx.doi.org/10.4121/uuid:c2c3b154-ab26-4b31-a0e8-8f2350ddac11>, https://data.4tu.nl/articles/dataset/BPI_Challenge_2013_closed_problems/12714476.
- [49] W. Steeman, BPI challenge 2013, incidents, 2013, <http://dx.doi.org/10.4121/uuid:500573e6-acc6-4b0c-9576-aa5468b10cee>, https://data.4tu.nl/articles/dataset/BPI_Challenge_2013_incidents/12693914.
- [50] W. Steeman, BPI challenge 2013, open problems, 2013, <http://dx.doi.org/10.4121/uuid:3537c19d-6c64-4b1d-815d-915ab0e479da>, https://data.4tu.nl/articles/dataset/BPI_Challenge_2013_open_problems/12688556.
- [51] B. van Dongen, BPI challenge 2017 - offer log, 2017, <http://dx.doi.org/10.4121/uuid:7e326e7e-8b93-4701-8860-71213edf0f8e>, https://data.4tu.nl/articles/dataset/BPI_Challenge_2017_-_Offer_log/12705737.
- [52] B. van Dongen, F.F. Borchert, BPI challenge 2018, 2018, <http://dx.doi.org/10.4121/uuid:3301445f-95e8-4ff0-98a4-901f1f204972>, https://data.4tu.nl/articles/dataset/BPI_Challenge_2018/12688355.
- [53] B. van Dongen, BPI challenge 2020, 2020, <http://dx.doi.org/10.4121/uuid:52fb97d4-4588-43c9-9d04-3604d4613b51>, https://data.4tu.nl/collections/BPI_Challenge_2020/5065541/1.
- [54] A.E. Johnson, T.J. Pollard, L. Shen, L.-w.H. Lehman, M. Feng, M. Ghassemi, B. Moody, P. Szolovits, L. Anthony Celi, R.G. Mark, MIMIC-III, a freely accessible critical care database, *Sci. Data* 3 (1) (2016) 1–9.
- [55] S.J.J. Leemans, L.L. Mannel, N. Sidorova, Significant stochastic dependencies in process models, *Inf. Syst.* 118 (2023) 102223.
- [56] M.M. de Leoni, F. Mannhardt, Road traffic fine management process, 2015, <http://dx.doi.org/10.4121/uuid:270fd440-1057-4fb9-89a9-b699b47990f5>, https://data.4tu.nl/articles/dataset/Road_Traffic_Fine_Management_Process/12683249.
- [57] F. Mannhardt, Sepsis cases - event log, 2016, <http://dx.doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460>, https://data.4tu.nl/articles/dataset/Sepsis_Cases_-_Event_Log/12707639.